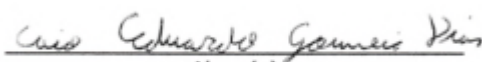


2º RELATÓRIO DE AÇÃO DE EXTENSÃO

IDENTIFICAÇÃO DO(A) ALUNO(A)		
Nome do Coordenador da equipe: Caio Eduardo Gouveia Dias		
Demais integrantes da equipe: Ana Clara Dantas da Luz, Bianca da Silva Barros e Sizenando Souza França		
Curso: Ciência da Computação		
Disciplina: Atividades Curriculares de Extensão I	Série: 2ª	
Professores: Dr. Nilton César de Paula e Dr. Jorge Marques Prates		
Local de Realização: Laboratório de Computação – Bloco G – UEMS Unidade de Dourados		
Público-Alvo: Jovens interessados na área de computação e programação.		
Período de Realização: Período de Realização: Março/2024 a Setembro/2024		
Carga Horária Anual da Ação: 136h		
TÍTULO		
Explorando a Linguagem e estrutura JavaScript.		
OBJETIVO(S)		
Apresentar a história e básico da Linguagem JavaScript, bem como ensinar as noções básicas de dados, operações lógicas e estruturas básicas, propondo, como finalização do curso, o desenvolvimento de programas de nível iniciante.		
ATIVIDADES DESENVOLVIDAS ATÉ NOVEMBRO		
Atividade	Mês	Carga horária
Realização do Curso “JavaScript {I, II, III, IV, V e VI}”	Junho	40h
Reunião 1 e 2	Março - Abril	4h
Reunião 3	Abril	1h
Reunião 4 e 5	Maio-Junho	4h
Desenvolvimento Aula 1	Maio	6h
Desenvolvimento Aula 2	Maio	10h
Desenvolvimento Aula 3	Maio	11h
Exercícios Elaborados	Junho	20h
Divulgação	Agosto	5h
Inscrição	Agosto	7h
Treinamento	Setembro/Vespertino	16h
Elaboração dos Relatórios	Outubro	12h
Total da carga horária (soma das cargas horárias das atividades)	136h	

** Anexe ao relatório os documentos de comprovação solicitados pelo professor. Todos os documentos devem estar em PDF.*

Dourados – MS, 08/10/2024


Aluno(a)

UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL – UEMS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Caio Eduardo Gouveia Dias, Ana Clara Dantas da Luz, Bianca da Silva Barros, Sizenando Souza França

Cita-se que os arquivos:

- Materiais Desenvolvidos e Aplicados;
- Materiais de Apoio;
- Exercícios Aplicados e Exemplos/Correções;
- Lista de Presença;
- Pastas e Arquivos Individuais dos Participantes;
- Fotos.

Estão anexados todos juntos ao final do arquivo, devidamente sinalizados e legendados, para melhor visualização do Relatório.

REUNIÃO 1 (21/03):

PARTICIPANTES:

Ana Clara Dantas da Luz;
Bianca da Silva Barros;
Caio Eduardo Gouveia Dias;
Sizenando Souza França.

DISCUSSÕES:

Reunião para estabelecer ideais de projeto e possíveis temas a ser trabalhado em cada ideia apresentada.
Chegando aos Pontos:

- Alfabetização Digital:
 - Word;
 - Excel;
 - Gmail;
 - Segurança de dois fatores.
- Treinamento JavaScript:
 - Estrutura Básica;
 - Tratamento de erros;
 - Comparação de Dados.

REUNIÃO 2 (04/04):

PARTICIPANTES:

Ana Clara Dantas da Luz;
Bianca da Silva Barros;
Caio Eduardo Gouveia Dias;
Sizenando Souza França.

DISCUSSÕES:

Tema estabelecido como “Explorando a Linguagem e estrutura JavaScript, ademais, fora feito a divisão de assuntos a serem trabalhados em cada aula, ou seja, definiu-se a ementa do curso.

RESULTADOS:

História da linguagem:

Ideia

Proposta

Surgimento

Desenvolvimento

Introdução ao Javascript:
Comparação de dados
Operadores lógicos
If else
Estrutura de repetição

Estrutura e funções:
Arrays
Switch
Função
Menu interativo

Aula prática:
Realização de atividades e programas utilizando o que foi passado durante as aulas do curso.

REUNIÃO 3 (11/04):

PARTICIPANTES:

Ana Clara Dantas da Luz;
Bianca da Silva Barros;
Caio Eduardo Gouveia Dias;
Sizenando Souza França.

DISCUSSÕES:

Discussões sobre o curso a ser realizado pelos participantes do grupo, predefinições de responsabilidade para as temáticas a serem trabalhadas em sala, início do planejamento das ferramentas de apoio.

RESULTADOS:

Curso: “JavaScript {I, II, III, IV, V e VI}”.

Plataforma: <https://cursos.onebitcode.com/area/vitrine>

Aulas:

- 1: Ana Clara Dantas da Luz;
- 2: Bianca da Silva Barros;
- 3: Sizenando Souza França;
- Aula Prática: Caio Eduardo Gouveia Dias.

REUNIÃO 4 (16/05):

PARTICIPANTES:

Ana Clara Dantas da Luz;
Bianca da Silva Barros;
Caio Eduardo Gouveia Dias;
Sizenando Souza França.

DISCUSSÕES:

Revisão dos Materiais de Aula 1,2 3.

RESULTADOS:

Material desenvolvido perante as discussões.

REUNIAO 5 (06/06):

PARTICIPANTES:

Ana Clara Dantas da Luz;
Bianca da Silva Barros;
Caio Eduardo Gouveia Dias;
Sizenando Souza França.

DISCUSSÕES:

Revisão da lista de exercícios planejada e proposta e revisão das atas a serem inseridas no relatório.

RESULTADOS:

Material desenvolvido perante as discussões.

DIVULGAÇÃO:

O projeto de divulgação iniciou-se no dia 03/08, por meio da abertura de uma página no site SYMPLA <https://www.sympla.com.br/>.

Onde o curso foi cadastrado e aberto a inscrições.

Link da página desenvolvida: <https://www.sympla.com.br/evento/explorando-a-linguagem-e-estrutura-javascript/2581786>

Posteriormente, no dia 05/08, o cartaz com as informações de inscrição e QR Code, que direcionava para o site SYMPLA foi desenvolvido:

Curso JavaScript

Explorando a Linguagem e estrutura JavaScript

Periodo de inscrição :

05 / Agosto

até

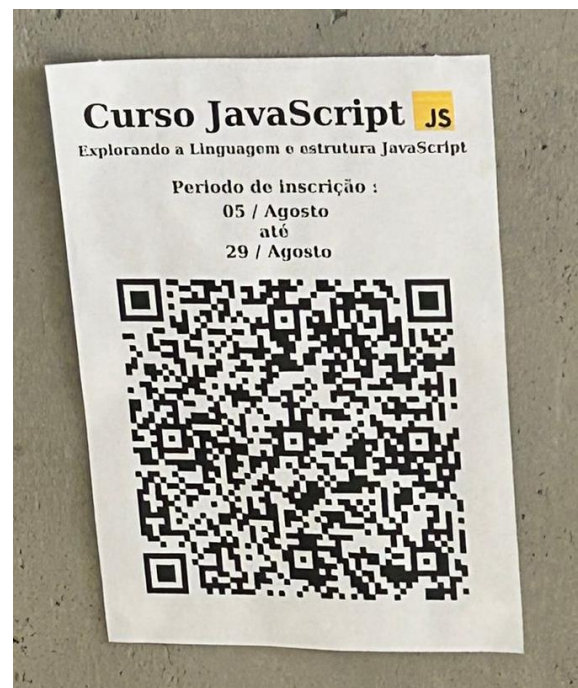
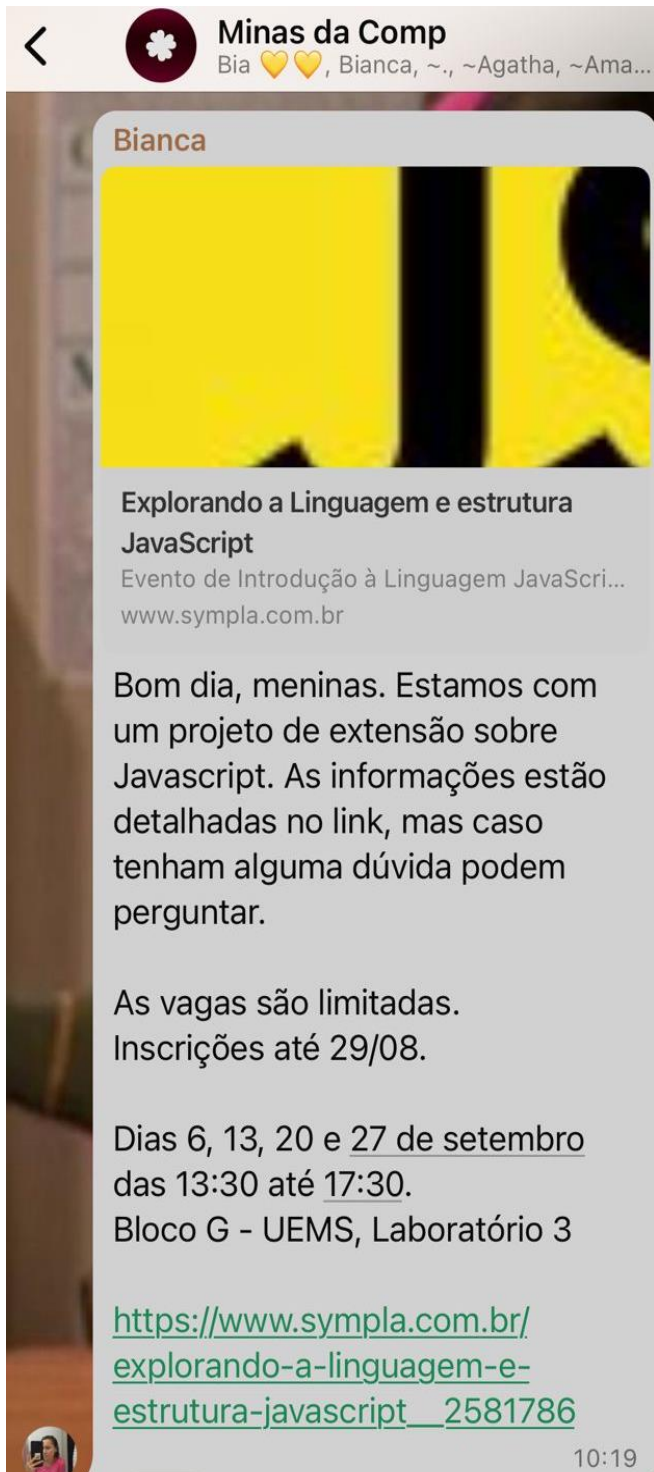
29 / Agosto



No dia 05/08 a divulgação presencial e online foi realizada, o curso foi apresentado de forma presencial na sala do 1º Ano de Ciência da Computação- UEMS, e posteriormente fora encaminhado uma mensagem no grupo de WhatsApp da mesma turma.

UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL – UEMS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Ademais, foi realizado o envio no grupo de WhatsApp “Minas da Comp”, e mais dois cartazes foram colados no Bloco G da UEMS, um no andar superior a frente do bebedouro e um abaixo da escada no andar inferior.



INSCRIÇÃO:

O período de Inscrição esteve aberto na SYMPLA do dia 05 de Agosto até 29 de Agosto. Foram ofertadas 25 vagas, o site obteve 139 visualizações na página de vendas e teve 96% de inscrições, ou seja, inicialmente supriu 24 das 25 vagas disponíveis.

● Explorando a Linguagem e estrutura JavaScript

Friday, 06/09/2024, 13h30 – Friday, 27/09/2024, 17h30 Bloco G - Laboratorio III , Dourados, MS

VISÃO GERAL

FORMULÁRIO DE INSCRIÇÃO

NÍVEIS DE ACESSO

INTEGRAÇÕES

CONFIGURAÇÕES ADICIO

DETALHES DO EVENTO

Status

● Publicado

Visualizações da página de vendas

139

Visibilidade

Público

Compartilhar

f

x

in

whatsapp

telegram

instagram

COPIAR LINK

https://www.sympla.com.br/explorando-a-linguage...

QR CODE



Baixar QR Code

☒	Nº ingresso	Nome	Sobrenome	Tipo de ingresso	Data compra
	TPST-J8-CVLK	Aline	Gauna de Andrade	Inscrições	09/08/2024
	TPQH-KR-FRUM	Ana	Júlia Stradioto Melo	Inscrições	08/08/2024
	TPU4-8V-X7K6	Beatriz	Lucas	Inscrições	10/08/2024
	TPSS-42-POYL	Bruno	Da Silva	Inscrições	09/08/2024
	TPSA-HC-3DYW	Bruno	Henrique	Inscrições	09/08/2024
	TQ4T-9B-DN2F	Claudia	de Almeida de Assis	Inscrições	15/08/2024
	TQ1T-26-VERS	Gabriela	Rabelo de Melo	Inscrições	13/08/2024
	TPR2-D8-7HX3	Guilherme	Barão	Inscrições	08/08/2024
	TQY3-3E-EJ02	Gustavo	Gareti	Inscrições	23/08/2024
	TQD0-U7-5F69	Isadora	Amal	Inscrições	08/08/2024
	TQ4Z-ED-NDQJ	Izabella	Miranda Silva	Inscrições	15/08/2024
	TPQH-P3-38N2	Jaine	Araujo dos Santos	Inscrições	08/08/2024
	TQDS-4Q-V0CE	João Pedro	Amaro Pippus	Inscrições	20/08/2024
	TPQW-GC-SRWT	Larissa Lury Yamaguc...	Bando	Inscrições	08/08/2024
	TQ1T-3Q-NPYJ	Leidinéia	da Cruz	Inscrições	13/08/2024
	TPST-HQ-DSV3	Leslen	Albino Vilhalta	Inscrições	09/08/2024
	TQ3K-VG-L7W1	Luan Matheus	Cordeiro	Inscrições	14/08/2024
	TQCX-Q9-7PPQ	Luís Felipe	Campos Araujo	Inscrições	20/08/2024
	TPQW-86-SJLN	Marcos Vinicius	Silva	Inscrições	08/08/2024
	TPZQ-MA-FUBR	Maria Eduarda	Cunha	Inscrições	07/08/2024
	TPP8-DK-EF0G	Miguel	Willame	Inscrições	07/08/2024
	TQ6R-PV-9AGP	Nicôle Lourdes	da Silva Gomes	Inscrições	16/08/2024
	TPQH-XU-J8B6	Rayanne	Antoniatti	Inscrições	08/08/2024
	TPQJ-DR-QTPN	Tiele	Carvalho	Inscrições	08/08/2024

Gerado em 16/09/2024 às 15h24

OBS: Apesar da Inscrição “Data Compra” é necessário salientar que as inscrições foram completamente gratuitas ofertadas pelo valor de R\$ 0.00 na SYMPLA

FINANCEIRO	
Vendas totais	Em processamento (pendentes)
R\$ 0,00	R\$ 0,00
Total a receber	Total recebido
R\$ 0,00	R\$ 0,00
Ticket médio (bruto)	Ticket médio (líquido)
R\$ 0,00	R\$ 0,00

TREINAMENTO:

Aula 1:

A aula 1 foi realizada no dia 06/09 e teve um total de 12 participantes.

Em um primeiro momento foi explicado como ligar o sistema, acessar o login comum do curso, e como encontrar o diretório de alunos com as pastas individuais já criadas, pastas essas onde cada atividade desenvolvida durante o projeto deveria ser salva.

Posteriormente uma rápida visualização dos temas a serem abordados foi realizada. E posteriormente aos momentos iniciais introduziram-se os conteúdos.

Os assuntos tratados foram:

- A história do JavaScript;
- Características Diferenciais;
- Aplicação no Mercado de Trabalho;
- Áreas Modernas da Aplicação do JavaScript;
- Diferenças Java e JavaScript;
- Conceitos Introdutórios do JavaScript, sendo eles:
 - Tipos de Dados;
 - Operadores;
 - Estruturas de Controle;
 - Estruturas de Repetição.

Após a explicação dos conceitos, os alunos foram dispensados para uma breve pausa, e com o retorno foi aplicado o exercício do dia a ser trabalhado pelos alunos, recebendo auxílio sempre que solicitado e futuramente, próximo ao horário da saída, a correção foi realizada no quadro, fazendo passo a passo da execução proposta.

Aula 2:

A aula 2 foi realizada no dia 13/09 e teve um total de 12 participantes.

Nessa Aula os conteúdos tratados foram;

- Arrays;
- Switch;
- Funções;

Todos os assuntos foram trabalhados com explicações e a aplicação de um exercício simples e rápido. Após a explicação dos dois primeiros tópicos, houve a liberação para um intervalo breve, e seguidamente, foi finalizado o último tópico. Por fim, houve a aplicação de um exercício maior e mais complexo, que abrangia todos os tópicos trabalhados anteriormente.

Aula 3:

A aula 3 foi realizada no dia 20/09 e teve um total de 10 participantes.

Os assuntos tratados foram:

- HTML
 - Introdução;
 - Estrutura Básica;
 - Tags Principais (Títulos; Parágrafos; Listas; Links; Forms);
- DOM;
 - Introdução;
 - Inserindo o arquivo HTML;
 - Conceitos-chave;
 - Método de Seleção;
 - Manipulação;
 - Alteração de Texto e/ou Atributos;
 - Adicionar ou Remover Elemento;
- Eventos;
- Entrada e Saída de Dados;

Os tópicos de HTML foram introduzidos nos primeiros horários, por meio da exibição de conteúdos e desenvolvimento prático de um a um. Para cada novo conceito, um exercício de pequeno porte e mais simplista foi aplicado para fixação do conteúdo. Após os momentos iniciais um breve intervalo foi aplicado, e ao retornar aplicaram-se os conteúdos de DOM e, para finalização do dia, um exercício de maior complexidade foi proposto, abrangendo todos os assuntos tratados no dia, que foi posteriormente, pouco antes da dispensa, corrigido em conjunto com os presentes.

Aula 4:

A última aula foi aplicada no dia 27/09 e teve uma presença de 08 participantes.

Os conteúdos de entrada de dados foram revisitados. Além da apresentação de aplicações HTML, em conjunto com o JavaScript, e suas versões sem o JavaScript, de modo a indicar o contraste na ausência de dinamismo e contato usuário-programa no segundo caso.

Após isso, foi iniciado os trabalhos de lógica e implementação do exercício final, repartido em partes menores. O exercício final abrangeu todos os conteúdos trabalhados nas 4 semanas de projeto, auxiliou na aprimoração da lógica de programação e proporcionou a conservação da linguagem na memória dos inscritos.

O exercício foi aplicado, os participantes tinham total acesso aos materiais de curso e aos ministrantes, todos os constituintes tiveram a capacidade de finalizar o trabalho.

Para Acesso das pastas individuais dos alunos, segue um link para o drive onde estão armazenadas:

<https://drive.google.com/drive/folders/1w4nbHiNZ7hdyhNVJ7ZEr3-hpeeSLKYAO?usp=sharing>

Para Acesso do Material de Aula disponibilizado para os alunos, segue o link do drive armazenamento:

https://drive.google.com/drive/folders/1y5PV8W8UOvu3Eeu2_0fCwSuZy4TUOcIW?usp=sharing

Em anexo a seguir, encontra-se:

- A lista de Presença;
- Materiais desenvolvidos, materiais auxiliares (quando existentes), resoluções e fotos, separadas por aula e anexadas cronologicamente.

Explorando a linguagem e estrutura JavaScript			
N. Inscrição	Nome	Sobrenome	
1	TPST-JB-CVJK	Aline	06/09/2024
2	TPQH-KR-FRUM	Ana	13/09/2024
3	TPU4-8V-X7K6	Beatriz	20/09/2024
4	TPSS-42-P0YL	Bruno	27/09/2024
5	TPSA-HC-3DYW	Bruno	
6	TQ4T-9B-DN2F	Claudia	
7	TQ1T-26-VERS	Gabriela	
8	TPR2-D8-7HX3	Guilherme	
9	TQY3-3E-EI02	Gustavo	
10	TQD0-U7-5F69	Isadora	
11	TQ4Z-ED-NDQJ	Izabella	
12	TPQH-P3-38N2	Jaine	
13	TQDS-4Q-V0CE	João Pedro	
14	TPQW-GC-SRWT	Larissa	
15	TQ1T-3Q-NPYJ	Leidineia	
16	TPST-HQ-DSV3	Leslen	
17	TQ3K-VG-L7W1	Luan Mathheus	
18	TQCK-Q9-7PPQ	Luís Felipe	
19	TPQW-86-SILN	Marcos Vinicius	
20	TPZQ-MA-FUBR	Maria Eduarda	
21	TPP8-DK-EF0G	Miguel	
22	TQ6R-PV-9AGP	Nicóle Lourdes	
23	TPQH-XU-J8B6	Rayanne	
24	TPQJ-DR-QTPN	Tiele	

ABAIXO ARQUIVOS RELACIONADOS A AULA 1, APLICADA DIA 06/09/2024

Em ordem os arquivos previamente planejados e o material de apoio desenvolvido.

Em seguida o exercício apresentado e sua resolução, por último as fotos referentes ao dia.

Introdução a JavaScript

Sumário

História.....	2
O que é JS?	2
Surgimento	2
Evolução	2
 Características.....	 3
Linguagem Script	3
POO Baseada em Protótipos:	3
Client-Side	3
Tipagem Mutável	4
 Relevância e Vantagens	 4
Web Development....	4
Simplicidade.....	4
Ecossistema Extensivo	4
Versatilidade.....	4
Performance e Eficiência	4
 Aplicações do JavaScript.....	 5
 Referencias Bibliográficas	 6

HISTÓRIA

O QUE É O JAVASCRIPT?

JavaScript trata-se de uma linguagem que permite a implementação de recursos básicos e avançados em páginas web, aplicativos, jogos, servidores, etc.

É uma linguagem que permite a criação de conteúdo que se atualiza dinamicamente, animações, controles de multimídia, menus suspensos, etc.. Em geral, o JavaScript possibilita uma melhor interação entre usuário e a página visitada.

Associada ao HTML e ao CSS, a linguagem compõe o 3º bloco de construção usados em sites modernos.

HTML: fornece a base dos sites, que é aprimorada por tecnologias como CSS e JavaScript;

CSS: responsável pela formatação, apresentação e layout;

JavaScript: utilizada no controle do comportamento de diferentes elementos.

SURGIMENTO

O JavaScript 1.0 teve seu lançamento em 1995, e a internet tratava-se de uma tecnologia recente. O primeiro navegador da web, o Mosaic, possuía 2 anos, e o HTML era a única ferramenta para programação na internet.

Criada por Brendan Eich, a pedido da empresa Netscape, o JavaScript surge visando validar formulários HTML, e chamava-se inicialmente LiveScript.

Em 1996, a Microsoft criou sua própria linguagem idêntica para ser usada no Internet Explorer, evidenciando a necessidade de padronizar a linguagem para diversos navegadores. A Netscape então regularizou a linguagem através da organização internacional ECMA, companhia especializada em padrões e regras.

EVOLUÇÃO

A linguagem passou por diversas etapas evolutivas, revisões e atualizações. Em sua fase inicial JavaScript possuía apenas funcionalidades básicas para manipulação de HTML e validação de formulários, sua evolução trouxe recursos de manipulação de DOM, animações, classes, módulos e outras funcionalidades modernas.

CARACTERÍSTICAS

LINGUAGEM DE SCRIPT

Como a nomenclatura indica, o JavaScript faz utilização do tipo de linguagem Script. Linguagens do tipo Script em geral são linguagens que dependem de um certo ambiente para serem executadas de maneira correta.

- É utilizada para automatizar tarefas e proporcionam o controle do funcionamento de um programa;
- Costumam ser mais simples e fáceis de utilizar;
- Ideal para iniciantes;
- É mais flexível, uma vez que, permite que desenvolvedores estendam e alterem funcionalidades de um programa sem alteração do código-fonte
- É, no geral, uma linguagem portátil, isto é, pode ser executado em diversos sistemas operacionais ao mesmo tempo, e sem alteração do código-fonte.

POO BASEADO EM PROTÓTIPOS

Diferindo-se de linguagens como Java ou C++, a herança em JavaScript é feita por meio de protótipos, diferindo dos citados que fazem uso de classes. A vantagem mais evidente é o fato que em JavaScript objetos podem herdar diretamente de outros objetos.

A orientação a objetos baseada em protótipos é um dos pilares que tornam JavaScript uma linguagem ideal para o desenvolvimento web. Ela oferece uma maneira flexível de criar e estender objetos.

CLIENT-SIDE

Existem dois tipos de linguagens no desenvolvimento web: as linguagens *client-side* e as linguagens *server-side*. Conhecidas também pelos nomes, front-end, para a linguagem client-side e back-end para as linguagens server-side.

JavaScript por sua vez é uma linguagem "client-side", linguagens onde o navegador é responsável pelo processamento. Ainda, Client-Side refere-se a tudo em um aplicativo web que é exibido ou ocorre no cliente (dispositivo do usuário final). Isso inclui o que o usuário vê, como texto, imagens, interface, junto com quaisquer ações que um aplicativo execute no navegador do usuário.

TIPAGEM MUTÁVEL

É possível alterar o tipo de variáveis ao longo do programa utilizando JavaScript, por exemplo, uma variável que inicialmente foi declarada como tipo float pode receber o tipo string no escopo do programa

Ainda se cita que, JavaScript permite a declaração de variáveis sem indicação de tipo, por meio da função *var*.

RELEVÂNCIA E VANTAGENS

Web Development:

JavaScript atualmente é fundamental para o dinamismo no desenvolvimento Web, uma vez que é responsável por aplicar uma interface cativante, adicionar interatividade, animações e diversas funcionalidades, não só estéticas, mas também funcionais.

Simplicidade:

É uma linguagem menos restritiva, e com fácil identificação de erros, além de apresentar uma estrutura simples, lhe tornando uma fácil porta de entrada para iniciantes e interessados na área, ainda, apresenta uma comunidade grande e diversos materiais e fóruns de auxílio.

Ecossistema Extensivo:

A linguagem apresenta um ecossistema diversificado, ou seja, ela é composta por diversas ferramentas, bibliotecas, frameworks e plataformas, o que a torna extremamente versátil.

Versatilidade:

Além de compatível com outras linguagens, PHP e Java, ainda é uma linguagem que não se restringe ao desenvolvimento web, sendo possível utilizá-la em outras áreas com a combinação de ferramentas ideais, indo desde de desenvolvimento de aplicativos até jogos.

Performance e Eficiência:

Ela é uma linguagem de boa velocidade, uma vez que, executa scripts de forma direta no navegador, sem a necessidade da passagem por um compilador primeiro. Além disso, a evolução de seus motores de execução otimizou a linguagem, de modo que, sua execução se torna constantemente mais rápida e eficiente, viabilizando sua utilização em aplicações de alto desempenho.

APLICAÇÕES DO JAVASCRIPT

De maneira prática, o JavaScript é utilizado para a aplicação do dinamismo na Web, citam-se funções como:

- Caixas de confirmação;
- Animação e outros efeitos especiais;
- Recursos de segurança, como senhas;
- Desenvolvimento de aplicações móveis.

Ainda, cita-se sua utilização em diversos sites e aplicações diariamente utilizados como:

- Conjunto Google: Gmail, Maps, Youtube, etc.;
- Netflix: Utilizando a linguagem tanto no back-end, quanto no front-end;
- Facebook;
- Twitter.

REFERENCIAS BIBLIOGRÁFICAS

Linguagem de Programação JavaScript: um breve histórico, CPT, disponível em:

<<https://www.cpt.com.br/cursos-informatica-desenvolvimentodesoftwares/artigos/linguagem-de-programacao-javascript-um-breve-historico#:~:text=JavaScript%20%C3%A9%20uma%20Linguagem%20de,com%20o%20nome%20Live%20Script&text=JavaScript%20%C3%A9%20uma%20Linguagem%20de%20Programa%C3%A7%C3%A3o%2C%20criada%20por%20Brendan%20Eich,Netscape%2C%20em%20meados%20de%201995>>

ADMIN. **Javascript: o que é, como surgiu e onde utilizar** - Resilia Educação. Resilia Educação.

Disponível em: <https://www.resilia.com.br/blog/javascript-o-que-e-como-surgiu-e-onde-utilizar/>

KEMPF, Rachel. **Conheça a história do JavaScript**. Azion Technologies. Disponível em:

<https://www.azion.com/pt-br/blog/a-historia-do-javascript/>

MARCELA, **A história do JavaScript: do passado ao presente**, Awari, disponível em:

<https://awari.com.br/a-historia-do-javascript-do-passado-ao-presente/?utm_source=blog&utm_campaign=projeto+blog&utm_medium=A%20hist%C3%B3ria%20do%20JavaScript:%20do%20passado%20ao%20presente>

Linguagem interpretada X Linguagem de script | Fórum Alura, Alura, disponível em:

<<https://cursos.alura.com.br/forum/topico-linguagem-interpretada-x-linguagem-de-script-193071#:~:text=Em%20resumo%2C%20quando%20falamos%20sobre,de%20aplica%C3%A7%C3%A3o%20de%20uma%20linguagem>>

Protótipos de objetos - Aprendendo desenvolvimento web | MDN, MDN Web Docs, disponível em:

<https://developer.mozilla.org/pt-BR/docs/Learn/JavaScript/Objects/Object_prototypes>

FAGUNDES, Marcio, **Web Design: Como funcionam HTML, CSS e JavaScript**, Almanaque Plus,

disponível em: <<https://almanaqueplus.com/web-design-como-funcionam-html-css-e-javascript/>>

Sumário

Dados	2
Tipos numéricos.....	2
Booleano	2
Indefinido	3
Null	3
String	4
Operadores	4
Operadores de atribuição	5
Operadores aritméticos.....	6
Operadores bit a bit.....	7
Operadores lógicos	8
Estrutura de controle	8
If...else	8
Estrutura de repetição.....	9
While	9
Do...while	9
For.....	9
For...in	10
For...of.....	10

Dados

Tipos numéricos

Em JavaScript, os números seguem o padrão IEEE 754, isso significa que pode representar números tão grandes quanto $\pm 1,7976931348623157 \times 10^{308}$ e tão pequenos quanto $\pm 5 \times 10^{-324}$.

Para declarar valores numéricos, basta atribuir os valores a uma variável.

Exemplos:

Inteiros

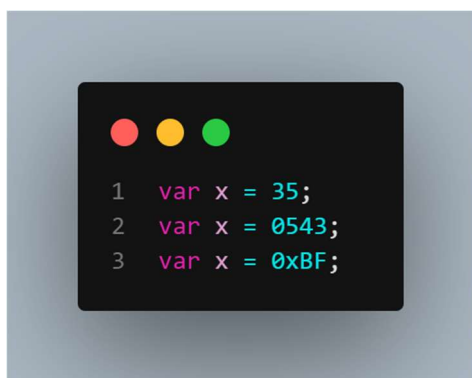


Figura 1: Atribuição para inteiros

Ponto flutuante



Figura 2: Atribuição para ponto flutuante

Booleano

Uma variável booleana pode ter apenas dois valores: *true* e *false*. Esses valores são geralmente usados como resultados de comparações e podem ser utilizados pelo usuário para testes ou para atributos binários. Eles são equivalentes ao uso de inteiros com valores 0 e 1 na linguagem C. O JavaScript converte automaticamente *true* para 1 e *false* para 0 quando necessário.

Exemplo:



```
1  var a = 14;
2  var b = 42;
3  var tr = (a == 14);
4  var fl = (a == b);
5
6  var int1 = tr+1;
7  var int2 = fl+1;
```

Figura 3: Atribuição booleano

Indefinido

Uma variável é considerada indefinida quando foi declarada, mas não possui nenhum valor atribuído a ela. Ao tentar acessar uma variável sem valor associado, o retorno será *"undefined"*.

Exemplo:



```
1  var marvin;
2  window.alert(marvin);
3
4
5  var text = "";
```

Figura 4: Atribuição indefinida

Null

O valor *null* representa a ausência de valor. Quando atribuímos *null* a um objeto ou variável, estamos indicando que essa variável ou objeto não possui um valor válido.

Ao usar o operador de igualdade `"=="`, JavaScript considera *null* e *undefined* como iguais. Isso é útil quando queremos verificar se um objeto possui determinado método, usando a comparação `(var.metodo == null)`. No entanto, se precisarmos diferenciar *null* de *undefined*, é recomendável usar o operador de identidade `"==="` que faz uma

comparação estrita. Portanto, para fins de comparação, *null* e *undefined* são iguais, mas não idênticos.

Exemplo:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains four lines of JavaScript code:

```
1 var vazio = null;
2 var ind;
3 var res = (vazio == ind);
4 var res1 = (vazio === ind);
```

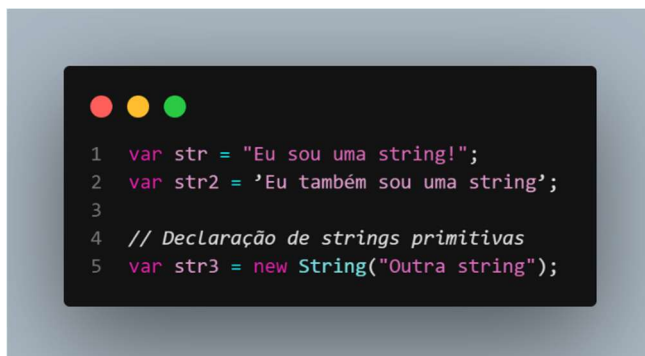
Figura 5: Atribuição Null

String

Em JavaScript, uma string pode ser tanto um tipo primitivo de dado quanto um objeto. No entanto, ao manipulá-la, parece que estamos lidando com objetos, pois as strings em JavaScript possuem métodos que podemos invocar para realizar diversas operações.

Essa impressão ocorre porque, ao criar uma string primitiva, o JavaScript também cria um objeto string e faz a conversão automática entre esses tipos quando necessário. Para declarar uma string, basta colocar uma sequência de caracteres entre aspas simples ou duplas.

Exemplo:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains five lines of JavaScript code:

```
1 var str = "Eu sou uma string!";
2 var str2 = 'Eu também sou uma string';
3
4 // Declaração de strings primitivas
5 var str3 = new String("Outra string");
```

Figura 6: Atribuição de string

Operadores

Operadores de atribuição

Atribui um valor ao operando à sua esquerda baseado no valor do operando à direita. O operador de atribuição básico é o igual (=), que atribui o valor do operando à direita ao operando à esquerda. Isto é, $x = y$ atribui o valor de y a x .

Alguns exemplos:

Nome	Operador simplificado	Significado
Atribuição	$x = y$	$x = y$
Atribuição de adição	$x += y$	$x = x + y$
Atribuição de subtração	$x -= y$	$x = x - y$
Atribuição de multiplicação	$x *= y$	$x = x * y$
Atribuição de divisão	$x /= y$	$x = x / y$
Atribuição de resto	$x \% = y$	$x = x \% y$
Atribuição exponencial	$x ** = y$	$x = x ** y$
Atribuição bit-a-bit por deslocamento á esquerda	$x << = y$	$x = x << y$
Atribuição bit-a-bit por deslocamento á direita	$x >> = y$	$x = x >> y$
Atribuição de bit-a-bit deslocamento á direita não assinado	$x >>> = y$	$x = x >>> y$
Atribuição AND bit-a-bit	$x \& = y$	$x = x \& y$
Atribuição XOR bit-a-bit	$x \wedge = y$	$x = x \wedge y$
Atribuição OR bit-a-bit	$x = y$	$x = x y$

Figura 7: Operadores de atribuição

Operadores de comparação

Um operador de comparação avalia seus operandos e retorna um valor booleano com base na veracidade da comparação. Os operandos podem ser números, strings, valores lógicos ou objetos.

Strings são comparadas conforme a ordem lexicográfica usando valores Unicode. Na maioria das situações, se os operandos são de tipos diferentes, o JavaScript tenta convertê-los para um tipo adequado. Isso geralmente resulta em uma comparação numérica.

As únicas exceções a essa regra são os operadores `===` e `!==`, que realizam comparações "estritas" de igualdade e desigualdade. Esses operadores não tentam converter os operandos para tipos compatíveis antes de verificar a igualdade.

A tabela abaixo descreve os operadores de comparação considerando o código a seguir:



Operador	Descrição	Exemplos que retornam verdadeiro
Igual (==)	Retorna verdadeiro caso os operandos sejam iguais.	3 == var1 "3" == var1 3 == '3'
Não igual (!=)	Retorna verdadeiro caso os operandos não sejam iguais.	var1 != 4 var2 != "3"
Estritamente igual (===)	Retorna verdadeiro caso os operandos sejam iguais e do mesmo tipo. Veja também Object.is e igualdade em JS.	3 === var1
Estritamente não igual (!==)	Retorna verdadeiro caso os operandos não sejam iguais e/ou não sejam do mesmo tipo.	var1 !== "3" 3 !== '3'
Maior que (>)	Retorna verdadeiro caso o operando da esquerda seja maior que o da direita.	var2 > var1 "12" > 2
Maior que ou igual (>=)	Retorna verdadeiro caso o operando da esquerda seja maior ou igual ao da direita.	var2 >= var1 var1 >= 3
Menor que (<)	Retorna verdadeiro caso o operando da esquerda seja menor que o da direita.	var1 < var2 "12" < "2"
Menor que ou igual (<=)	Retorna verdadeiro caso o operando da esquerda seja menor ou igual ao da direita.	var1 <= var2 var2 <= 5

Figura 8: Operadores de comparação

Operadores aritméticos

Operadores aritméticos utilizam valores numéricos, sejam literais ou variáveis, como operandos e retornam um único valor numérico. Os operadores aritméticos básicos incluem soma (+), subtração (-), multiplicação (*) e divisão (/).

Quando usados com números de ponto flutuante, esses operadores funcionam da mesma maneira que na maioria das linguagens de programação (observando, por exemplo, que a divisão por zero resulta em NaN, significa Not-A-Number – não é um número).

Exemplo:



Operador	Descrição	Exemplo
Módulo (%)	Operador binário. Retorna o inteiro restante da divisão dos dois operandos.	12 % 5 retorna 2.
Incremento (++)	Operador unário. Adiciona um ao seu operando. Se usado como operador prefixado (++x), retorna o valor de seu operando após a adição. Se usado como operador pósfixado (x++), retorna o valor de seu operando antes da adição.	Se x é 3, então ++x define x como 4 e retorna 4, enquanto x++ retorna 3 e somente então, define x como 4.
Decremento (--)	Operador unário. Subtrai um de seu operando. O valor de retorno é análogo àquele do operador de incremento.	Se x é 3, então --x define x como 2 e retorna 2, enquanto x-- retorna 3 e somente então, define x como 2.
Negação (-)	Operador unário. Retorna a negação de seu operando.	Se x é 3, então -x retorna -3.
Adição (+)	Operador unário. Tenta converter o operando em um número, sempre que possível.	+"3" retorna 3.+true retorna 1.
Operador de exponenciação (**) Experimental	Calcula a base elevada à potência do expoente, que é, baseexpoente	2 ** 3 retorna 8.10 ** -1 retorna 0.

Figura 9: Operadores aritméticos

Operadores bit a bit

Operadores bit a bit tratam seus operandos como sequências de 32 bits (zeros e uns), ao invés de números decimais, hexadecimais ou octais. Por exemplo, o número decimal nove é representado em binário como 1001. Esses operadores realizam operações sobre essas representações binárias, mas retornam valores numéricos padrão em JavaScript.

Operador	Expressão	Descrição
AND	a & b	Retorna um 1 para cada posição em que os bits da posição correspondente de ambos operandos sejam uns.
OR	a b	Retorna um 0 para cada posição em que os bits da posição correspondente de ambos os operandos sejam zeros.
XOR	a ^ b	Retorna um 0 para cada posição em que os bits da posição correspondente são os mesmos. [Retorna um 1 para cada posição em que os bits da posição correspondente sejam diferentes.]
NOT	~ a	Inverte os bits do operando.
Deslocamento à esquerda	a << b	Desloca a em representação binária b bits à esquerda, preenchendo com zeros à direita.
Deslocamento à direita com propagação de sinal	a >> b	Desloca a em representação binária b bits à direita, descartando bits excedentes.
Deslocamento à direita com preenchimento zero	a >>> b	Desloca a em representação binária b bits à direita, descartando bits excedentes e preenchendo com zeros à esquerda.

Figura 10: Operadores bit a bit

Operadores lógicos

São usados com valores booleanos e, nesse contexto, retornam um valor booleano. No entanto, os operadores `&&` e `||` na verdade retornam o valor de um dos operandos fornecidos, o que significa que, se esses operadores forem usados com valores não booleanos, eles podem retornar um valor não booleano. A tabela a seguir descreve os operadores lógicos.

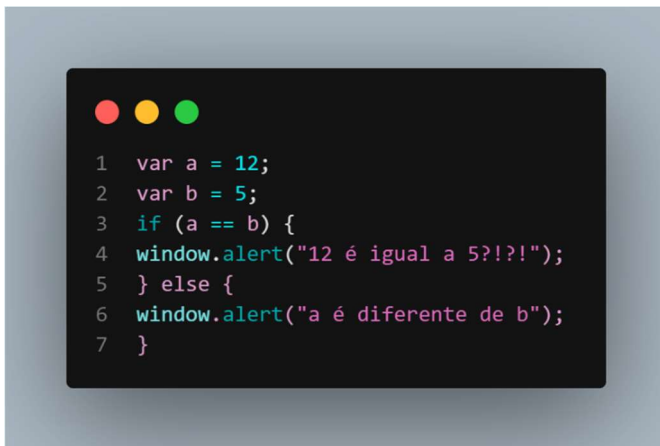
Operador	Utilização	Descrição
AND lógico (&&)	<code>expr1 && expr2</code>	(E lógico) - Retorna <code>expr1</code> caso possa ser convertido para falso; senão, retorna <code>expr2</code> . Assim, quando utilizado com valores booleanos, <code>&&</code> retorna verdadeiro caso ambos operandos sejam verdadeiros; caso contrário, retorna falso.
OU lógico ()	<code>expr1 expr2</code>	(OU lógico) - Retorna <code>expr1</code> caso possa ser convertido para verdadeiro; senão, retorna <code>expr2</code> . Assim, quando utilizado com valores booleanos, <code> </code> retorna verdadeiro caso ambos os operandos sejam verdadeiros; se ambos forem falsos, retorna falso.
NOT lógico (!)	<code>!expr</code>	(Negação lógica) Retorna falso caso o único operando possa ser convertido para verdadeiro; senão, retorna verdadeiro.

Figura 11: Operadores lógicos

Estrutura de controle

If...else

A estrutura *if* é usada quando se deseja verificar se determinada expressão é verdadeira ou não, e executar comandos específicos para cada caso.

A screenshot of a code editor with a dark background and light-colored text. The code is written in JavaScript and uses syntax highlighting. It defines two variables, 'a' and 'b', and then uses an if-else statement to check if 'a' is equal to 'b'. If true, it shows an alert message; if false, it shows another alert message.

```
1 var a = 12;  
2 var b = 5;  
3 if (a == b) {  
4   window.alert("12 é igual a 5?!?!");  
5 } else {  
6   window.alert("a é diferente de b");  
7 }
```

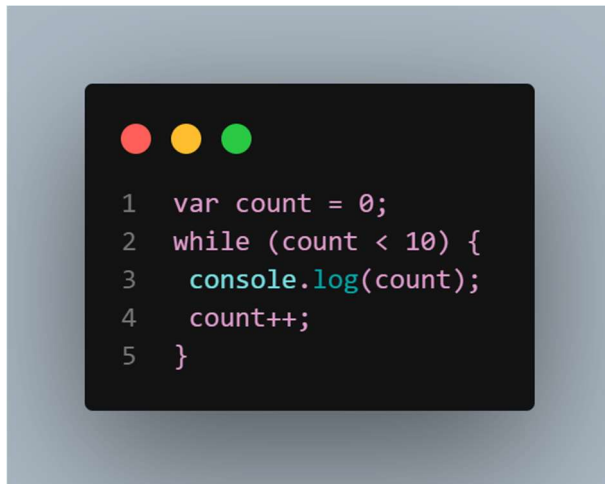
Figura 12: Estrutura de controle

Assim, se a expressão for avaliada como verdadeira, o primeiro bloco de comandos é executado. Caso seja avaliada como falsa, o bloco de comandos que segue o *else* será executado.

Estrutura de repetição

While

São usados quando se deseja que uma sequência de ações seja executada apenas no caso de a expressão de condição ser válida. Assim, primeiro a expressão é testada, para depois o conteúdo do laço ser executado ou não. Exemplo:

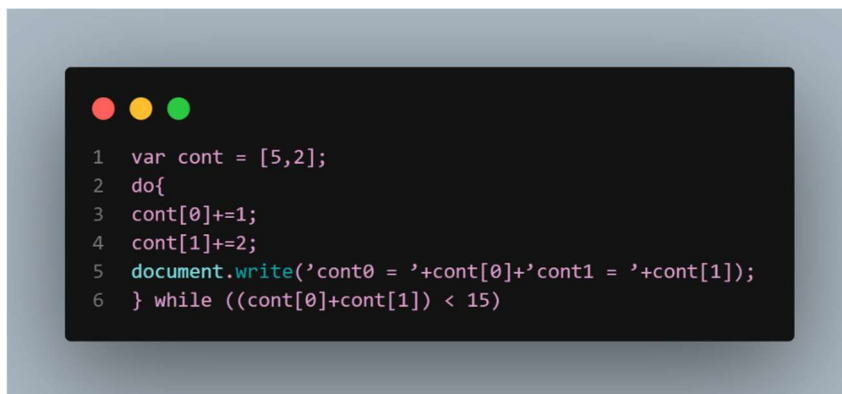
A code editor window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It contains five lines of JavaScript code:

```
1 var count = 0;
2 while (count < 10) {
3   console.log(count);
4   count++;
5 }
```

Figura 13: Estrutura de repetição while

Do ... while

Primeiro executa o conteúdo do laço uma vez e, depois disso, realiza o teste da expressão para decidir se continuará executando o laço ou irá seguir o resto do programa. Exemplo:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It contains six lines of JavaScript code:

```
1 var cont = [5,2];
2 do{
3   cont[0]+=1;
4   cont[1]+=2;
5   document.write('cont0 = '+cont[0]+'cont1 = '+cont[1]);
6 } while ((cont[0]+cont[1]) < 15)
```

Figura 14: Estrutura de repetição do..while

For

Nesse tipo de laço, a inicialização, o teste e a atualização são as três manipulações fundamentais de uma variável de laço. A instrução *for* codifica cada uma dessas três

manipulações como uma expressão e torna essas expressões uma parte explícita da sintaxe do laço.

Exemplo:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains four lines of JavaScript code:

```
1 var cont = [5,2,3];  
2 for(var i=0 ; i < 3 ; i++) {  
3   cont[i]++;  
4 }
```

Figura 15: Estrutura de repetição *for*

For...in

Existe uma segunda forma de se utilizar os laços *for* para percorrer propriedades de um objeto. Se *for* é avaliada como *null* ou *undefined*, o interpretador pula o laço e passa para a instrução seguinte. Se a expressão é avaliada como um valor primitivo, esse valor é convertido em seu objeto empacotador equivalente. Caso contrário, a expressão já é um objeto.

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It contains two lines of JavaScript code:

```
1 for(var p in o)  
2   console.log(o[p]);
```

Figura 16: Estrutura de repetição *for..in*

For...of

O laço *for...of* itera pelos valores de muitos tipos de iteráveis, incluindo *arrays* e tipos especiais de coleções, como *Set* e *Map*. Para cada valor no objeto iterável, as instruções dentro do laço são executadas.

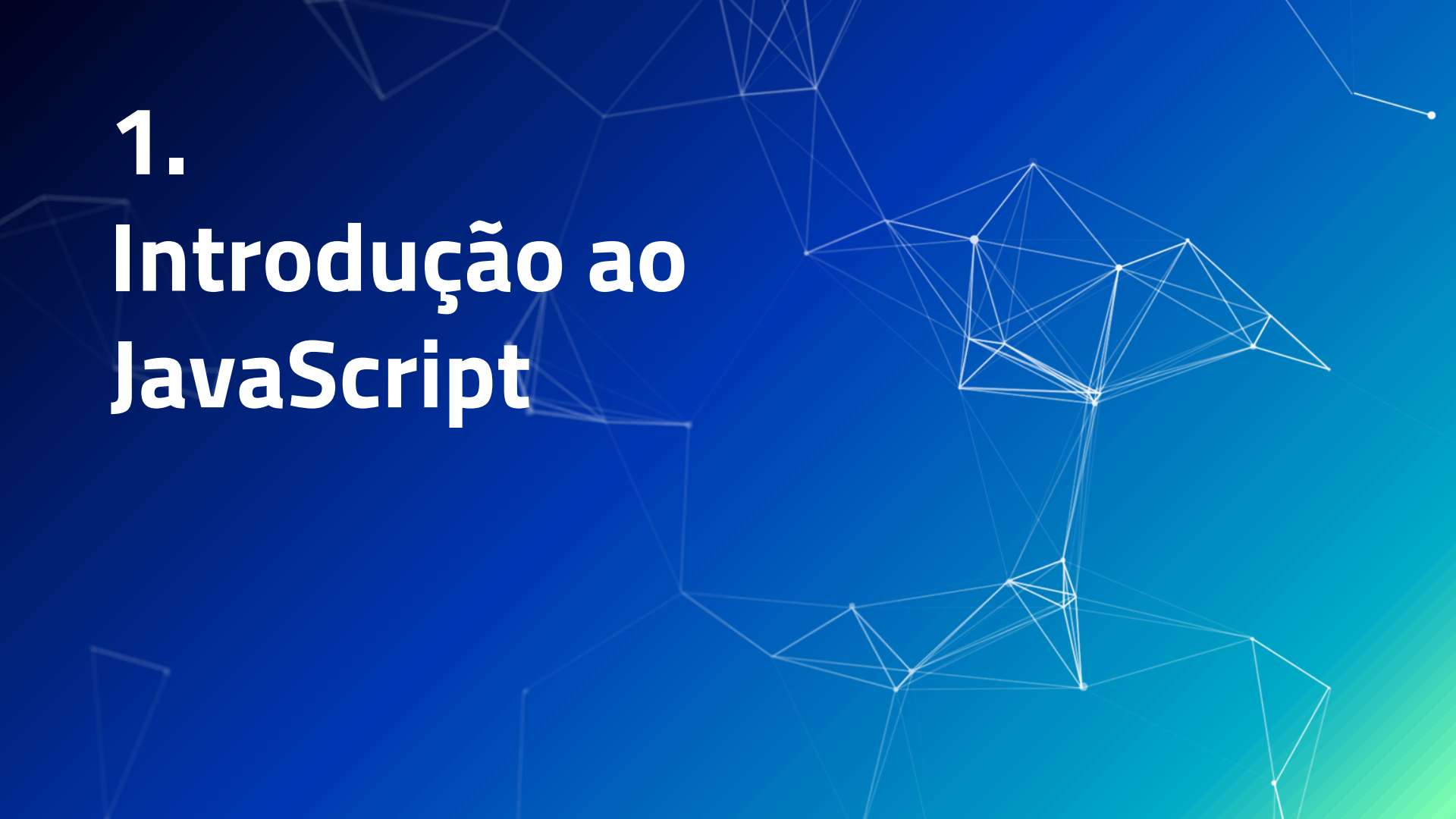
Enquanto o *for...in* interage com o nome das propriedades, o *for...of* interage com o valor das propriedades. O exemplo a seguir mostra um exemplo utilizando *array*.



Figura 17: Estrutura de repetição for..of

Explorando a Linguagem e estrutura JavaScript.

1. Introdução ao JavaScript

The background of the slide is a deep blue gradient that transitions into a lighter blue and green at the bottom right. Overlaid on this background is a complex network of thin white lines connecting small white dots, creating a geometric, wireframe-like structure that resembles a molecular model or a network diagram. The lines and dots are scattered across the frame, with some forming more dense clusters than others.

O que é JavaScript?

- JavaScript é uma linguagem de programação essencial para o desenvolvimento web.
- Permite a criação de conteúdos dinâmicos, como animações, controles de multimídia, e menus interativos.
- É integrável com os outros 2 blocos fundamentais de desenvolvimento de sites.
 - HTML: Estrutura o conteúdo da web.
 - CSS: Cuida do design e layout.
 - JavaScript: Controla o comportamento dos elementos da página, adicionando interatividade.

2. Surgimento do JavaScript

The background of the slide is a blue gradient, transitioning from a darker blue on the left to a lighter blue and green on the right. Overlaid on this background is a complex network of white lines and dots, forming various geometric shapes like triangles and polygons, some of which are interconnected to form larger, more complex structures. This network-like pattern is reminiscent of a molecular structure or a digital network.

Surgimento do JavaScript

Lançamento

Criado em 1995 por Brendan Eich para a Netscape, inicialmente chamado de LiveScript.

Motivação

Validar formulários HTML e melhorar a interatividade nas páginas web.

Padronização

Com a popularização, a Microsoft criou uma versão similar para o Internet Explorer, levando à necessidade de padronização pela ECMA (ECMAScript).

3.

Evolução da Linguagem



Evolução da Linguagem

- JavaScript começou com funcionalidades básicas para manipulação de HTML e validação de formulários.
- Ao longo do tempo, foram introduzidas novas funcionalidades, como:
 - Manipulação do DOM (Document Object Model).
 - Criação de animações e efeitos visuais.
 - Introdução de classes e módulos para estruturação de código

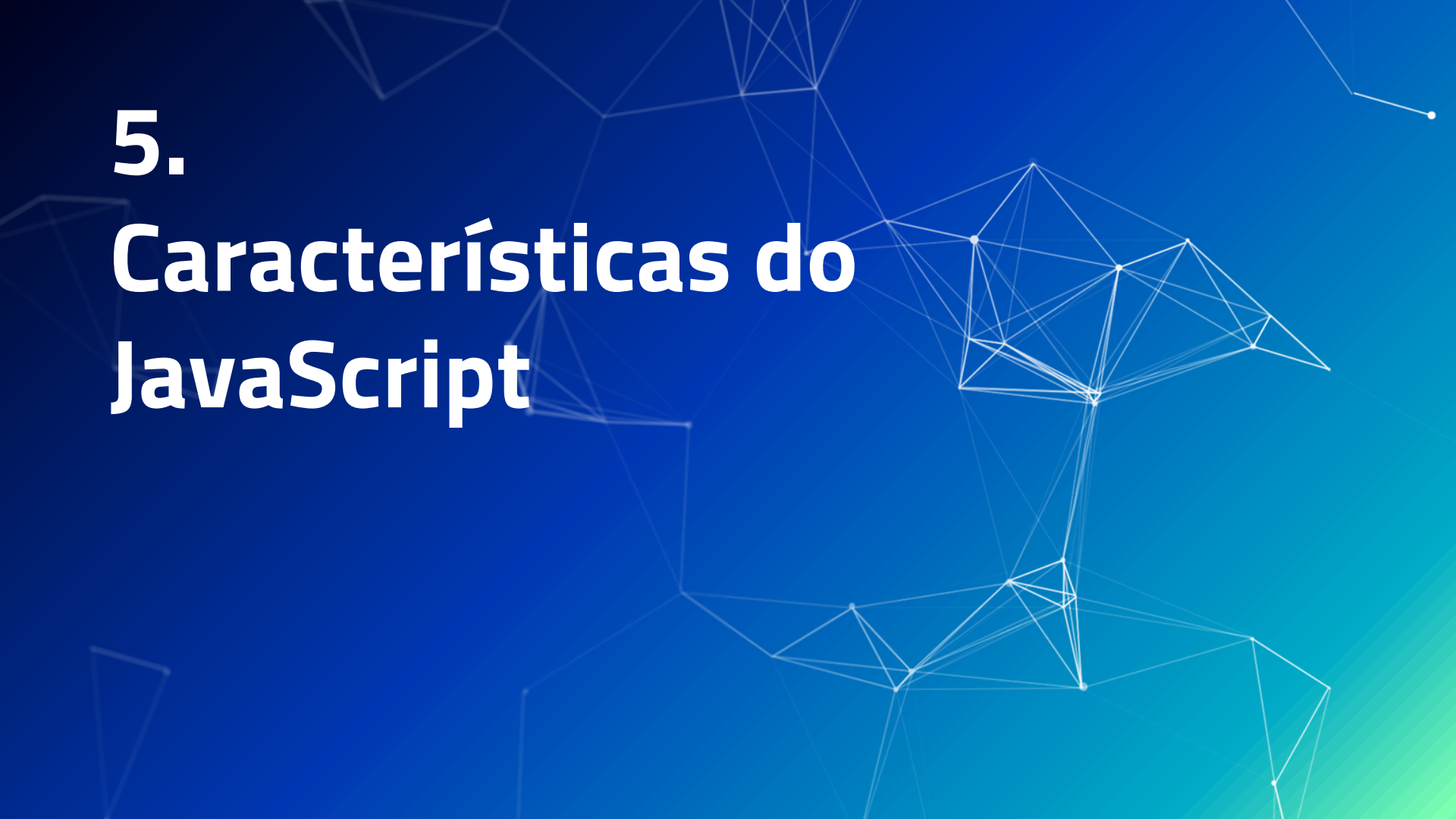


4. Linguagem de Script

■ JavaScript é uma linguagem de script, executada em um ambiente específico, como navegadores.

■ Vantagens

- Automatização: Facilita a automatização de tarefas e controle do comportamento de programas.
- Simplicidade: Mais fácil de aprender e utilizar, ideal para iniciantes.
- Flexibilidade: Permite a extensão e alteração de funcionalidades sem modificar o código-fonte original.
- Portabilidade: Pode ser executado em diferentes sistemas operacionais sem a necessidade de modificações.



5.

Características do JavaScript

POO Baseado em Protótipos

- **Diferença:** Diferente de linguagens como Java e C++, que usam classes para herança, JavaScript usa protótipos.
- **Vantagem:** Objetos podem herdar diretamente de outros objetos, oferecendo mais flexibilidade no desenvolvimento.
- **Importância:** A POO baseada em protótipos é fundamental para o desenvolvimento web em JavaScript.

Client-Side

JavaScript é executado no lado do cliente, ou seja, no navegador do usuário.

■ Funções Client-Side:

- Processamento de dados localmente, sem necessidade de comunicação com o servidor.
- Controle de tudo que o usuário vê e interage na página (texto, imagens, interface).
- Execução de ações no navegador, como validação de formulários e manipulação do DOM.

Tipagem Mutável

- **Flexibilidade:** JavaScript permite alterar o tipo de variáveis durante a execução do programa.
- **Exemplo:** Uma variável pode ser inicialmente do tipo float e posteriormente ser alterada para string.
- **Declaração:** Variáveis podem ser declaradas sem um tipo específico, utilizando a palavra-chave "var".

6.

Relevância e Vantagens



Relevância no Desenvolvimento Web

- **Dinamismo:** Essencial para criar interfaces interativas e dinâmicas na web.
- **Simplicidade:** Estrutura simples e fácil de aprender, ideal para iniciantes.
- **Comunidade:** Grande quantidade de recursos, fóruns e bibliotecas disponíveis.
- **Ecossistema:** Extensa variedade de ferramentas, bibliotecas e frameworks que ampliam as capacidades do JavaScript.

Versatilidade e Performance

- Versatilidade: Além do desenvolvimento web, JavaScript pode ser utilizado em diversas outras áreas, como:
 - Desenvolvimento de aplicativos móveis.
 - Criação de jogos.
- Performance: Executa diretamente no navegador, sem necessidade de compilação, proporcionando alta velocidade e eficiência.

6.

Aplicações do JavaScript

The background of the slide is a blue gradient, transitioning from a darker blue on the left to a lighter blue and green on the right. Overlaid on this background is a complex network of white lines and dots, forming various geometric shapes like triangles and polygons. Some of these shapes are more prominent than others, creating a sense of depth and connectivity.

Aplicações Práticas

■ Funcionalidades:

- Criação de caixas de confirmação.
- Implementação de animações e efeitos especiais.
- Recursos de segurança, como validação de senhas.

Aplicações Práticas

■ Exemplos de Uso:

- Google: Gmail, Maps, YouTube.
- Netflix: Utilização tanto no back-end quanto no front-end.
- Redes Sociais: Facebook e Twitter.

Exercício Aplicado:

1. Implementar um código que dado a entrada de um número natural “x”, dê a saída de x por extenso.
(x estará apenas na casa das unidades e dezenas [0 á 99])
exemplo:

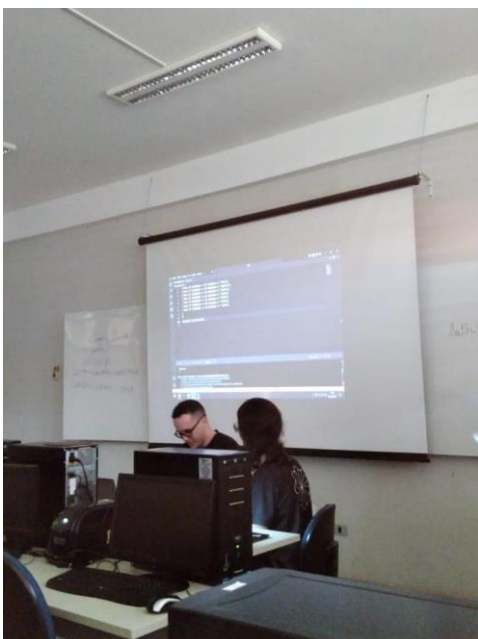
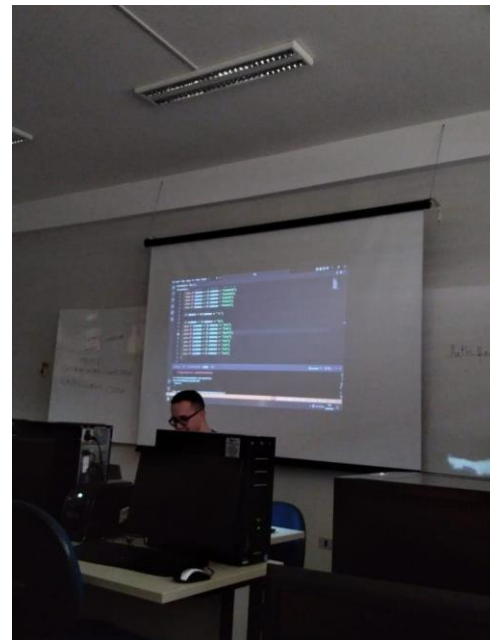
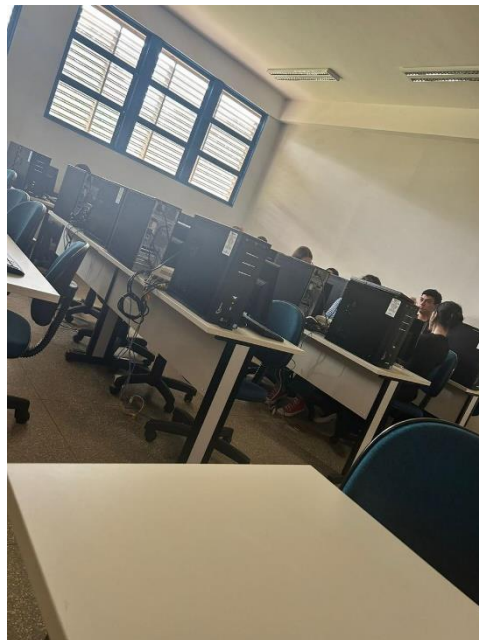


Resolução Apresentada:

```
1 let number = 45;
2
3 let dezena = Math.floor(number / 10);
4 let unidade = number % 10;
5
6 let extenso = "";
7
8 if (dezena === 1) {
9     if (dezena === 1) extenso = "onze";
10    if (dezena === 2) extenso = "doze";
11    if (dezena === 3) extenso = "treze";
12    if (dezena === 4) extenso = "quatorze";
13    if (dezena === 5) extenso = "quinze";
14    if (dezena === 6) extenso = "dezeseis";
15    if (dezena === 7) extenso = "dezeseite";
16    if (dezena === 8) extenso = "dezoito";
17    if (dezena === 9) extenso = "dezenove";
18 } else {
```

```
19     if (dezena === 2) extenso = "vinte";
20     else if (dezena === 3) extenso = "trinta";
21     else if (dezena === 4) extenso = "quarenta";
22     else if (dezena === 5) extenso = "cinquenta";
23     else if (dezena === 6) extenso = "sessenta";
24     else if (dezena === 7) extenso = "setenta";
25     else if (dezena === 8) extenso = "oitenta";
26     else if (dezena === 9) extenso = "noventa";
27
28     if (dezena !== 0) extenso += " e ";
29
30     if (unidade === 1) extenso += "um";
31     else if (unidade === 2) extenso += "dois";
32     else if (unidade === 3) extenso += "tres";
33     else if (unidade === 4) extenso += "quatro";
34     else if (unidade === 5) extenso += "cinco";
35     else if (unidade === 6) extenso += "seis";
36     else if (unidade === 7) extenso += "sete";
37     else if (unidade === 8) extenso += "oito";
38     else if (unidade === 9) extenso += "nove";
39 }
40
41 console.log(`O numero ${number} por extenso é: ${extenso}`);
42
```


Fotos:



ABAIXO ARQUIVOS RELACIONADOS A AULA 2, APLICADA DIA 13/09/2024

Em ordem os arquivos previamente planejados.

Em seguida o exercício apresentado e sua resolução, por último as fotos referentes ao dia.

Sumário

Array	2
Introdução	2
Declaração	2
Manuseando	3
Principais métodos	3
Push e Pop.....	3
Unshift e Shift.....	4
Splice.....	4
Concat	4
forEach	5
Sugestão de exercício.....	5
Switch	6
Introdução	6
Declaração	6
Diferenciais.....	7
Sugestão de exercício.....	8
Funções	8
Introdução	8
Declaração e uso	8
Operador Spread.....	9
Introdução.....	9
Sintaxe e uso	9
Arrow Functions	10
Introdução.....	10
Sintaxe e uso	10
Método Reduce	11
Sugestão de exercício.....	12
Exercício Final	14

Curiosidade: o uso do var foi descontinuado pois sua declaração funcionava em todo programa e não a partir da linha a qual foi declarada, ou seja, se você declarar uma variável dentro de uma função, ela funcionará para todas as funções externas, além disso, o var permitia redeclarar a mesma variável em outra parte do programa, o uso do let para declaração se tornou uma escolha mais segura para aqueles que desejam criar programas com maior controle e segurança. Resumidamente, var possui escopo global enquanto let um escopo de bloco (let significa “deixe”, essa escolha queria remeter à “Deixe a variável nesse escopo”).

Array

Introdução

Um array é, como em toda linguagem de programação, uma forma de armazenar mais de um valor em apenas uma variável, sendo acessível através de índices é possível armazenar, ler e alterar valores.

Diferente da linguagem C onde é necessário declarar o tipo do vetor, em Javascript, sua declaração ainda é flexível, podendo armazenar os mais diversos tipos de dados misturados, seja inteiro, float, string e até mesmo outros array e objetos.

Declaração

A declaração de um array é semelhante as demais estrutura de dados, utilizando-se de colchetes para indicar a criação de um vetor:



Figura 1: Declaração de um vetor

Se necessário, pode declarar a variável já com os valores desejados:



Figura 2: Vetor declarado com valores

Diferente da linguagem C onde seria necessário declarar um vetor para cada tipo primitivo, aqui podemos colocar todos juntos e funcionará perfeitamente.

Um diferencial é o uso de chaves ao inserir valores no seu vetor, basta dar um nome para o índice e passar o seu valor desejado:

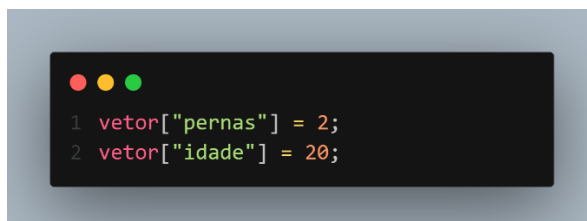


Figura 3: Usando chaves como índices no vetor

Manuseando

A maneira de acessar os valores é semelhante em todas as linguagens, você insere um número indicando o índice do valor a ser acessado entre colchetes ou, caso você tenha declarado chaves, colocar a sua chave entre os colchetes:

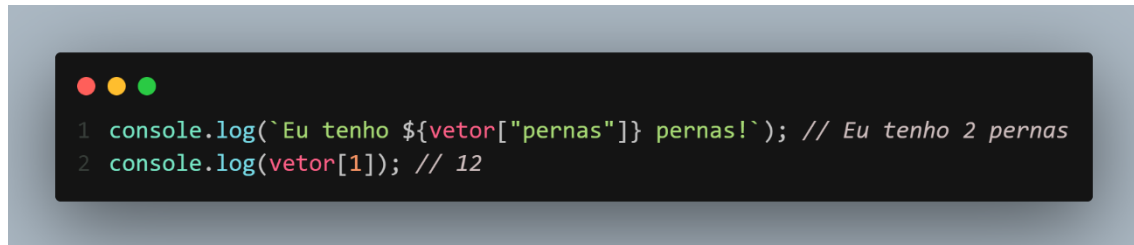


Figura 4: Acesso do array por chave e por índice

Principais métodos

Para facilitar a vida do programador, o Javascript disponibiliza uma série de métodos para administrarmos esses array, para servir de comparação sobre a eficácia, em C você teria que implementar todos esses métodos um por um manualmente para fazer exatamente a mesma coisa.

Os métodos nada mais são funções pré-definidas que realizam alguma tarefa específica, os métodos estarão disponíveis em diversas estruturas de dados além dos array, para usar, basta clicar ponto (.) para a IDE mostrar todos os métodos disponíveis:

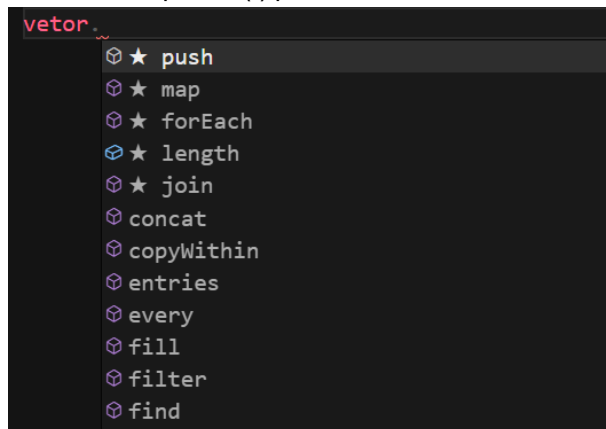


Figura 5: IDE mostrando métodos disponíveis

A seguir será mostrado as principais delas e como elas funcionam.

Push e Pop

O método push realiza a inserção no final de um array, você não precisa saber qual o índice ou quantos elementos existem no array:



Figura 6: Inserindo valor no final da lista

Já o método `pop` faz o contrário, ele remove o último elemento do array:



Figura 7: Removendo elemento do final da lista

Unshift e Shift

O método `unshift` também faz uma inserção, porém será feito no início do vetor, você não precisa se preocupar com o restante dos elementos pois eles serão “empurrados” para a direita automaticamente, e o método `shift` é autoexplicativo, ele faz o oposto removendo o primeiro elemento do array:



Figura 8: Inserindo e removendo elemento no início do array

Splice

Esse método remove elementos dentro de um intervalo desejado, você escolhe no primeiro parâmetro qual índice que deseja começar e escolhe no segundo parâmetro quantos elementos devem ser apagados a partir do início escolhido:

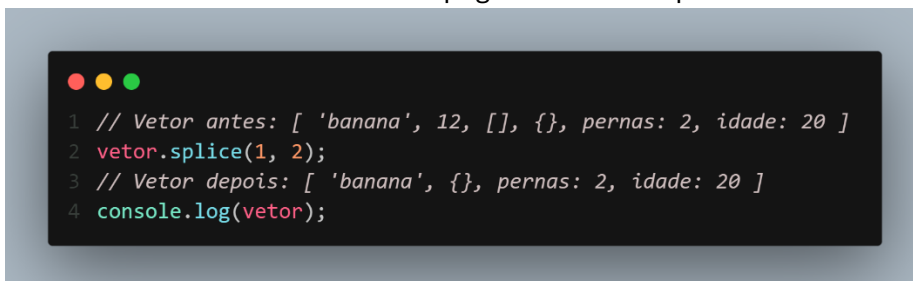


Figura 9: Removendo dois elementos a partir do índice 1

Perceba que o elemento do índice 1, “12”, e o array “[]” foram apagados, exatamente dois elementos como escolhido.

Caso você não passe nenhum valor no segundo parâmetro, todos os elementos serão apagados a partir do índice de início escolhido.

Concat

Esse método concatena dois ou mais array, isso significa que se você tem dois ou mais arrays e deseja juntar eles em um array só, isso é possível. Um detalhe é que esse método retornará um array, ou seja, ele não vai alterar nenhum array envolvido no processo, por isso é necessário que você crie outro array para receber esses array concatenados:

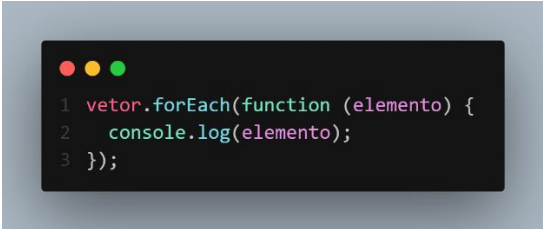


```
1 let vetor2 = [192];
2 let vetor3 = vetor.concat(vetor2);
3 // [ 'banana', 12, [], {}, 192 ]
```

Figura 10: Concatenação de dois vetores

forEach

O método `forEach` provavelmente é o que mais será utilizado, ele substitui o uso do `for` para iterações completas do array, não sendo necessário passar o índice inicial ou até qual índice a repetição deve ocorrer. Nela, basta passar uma função contendo o que você deseja que deva ser feito para cada elemento desse vetor. Esse método usa o conceito de “High Order Function”, que é o ato de uma função ter outra função no seu parâmetro, ou seja, no método `forEach` sempre terá uma função em seu parâmetro:



```
1 vetor.forEach(function (elemento) {
2   console.log(elemento);
3 });
```

Figura 11: Exibição de cada elemento do vetor

Perceba que no parâmetro da função do método `forEach` está sendo usado “elemento”, esse parâmetro simboliza um único elemento do array, e toda vez que a função for executada e ir para a próxima iteração, esse elemento será o próximo do array, será incrementado automaticamente até que não exista mais elementos.

Sugestão de exercício

O exercício proposto será de inserção de 4 notas de um aluno num vetor e em seguida calcular a média aritmética e dizer se o aluno foi aprovado ou não (média maior que 6).
Resolução:

```

1 let notas = [6, 7.6, 9, 1];
2
3 // Calcula a média das notas
4 let soma = 0;
5 notas.forEach(function (nota) {
6     soma += nota;
7 });
8
9 let media = soma / notas.length;
10
11 // Verifica se aluno passou
12 let passou = media > 6 ? true : false;
13
14 // Exibe a média das notas e o resultado
15 if (passou) {
16     console.log(`A média das notas é: ${media.toFixed(2)}, o aluno passou`);
17 } else {
18     console.log(`A média das notas é: ${media.toFixed(2)}, o aluno reprovou`);
19 }
20

```

Figura 12: Resolução do exercício de arrays

Switch

Introdução

O switch assim como o if, é uma estrutura de controle de fluxo, mas em alguns casos ela se torna mais limpa do que se usar vários IFs e ELSEs, como por exemplo no uso de menus. Ele avalia uma variável e compara qual caso se aplica aquela variável.

Declaração

Sua estrutura é idêntica a linguagem C, possui o mesmo campo da variável, possui os mesmos cases e o mesmo default, não podendo esquecer de colocar um break em cada case (exceto no default):

```

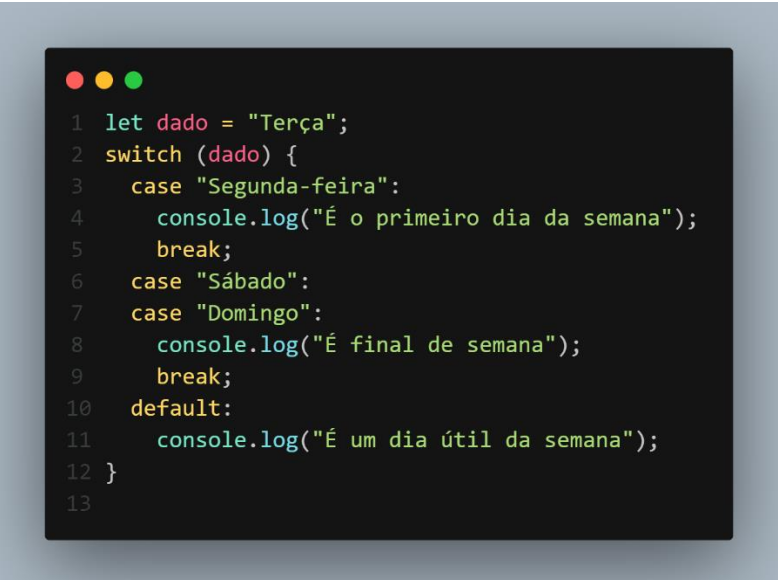
1 let dado;
2 switch (dado) {
3     case 1:
4         break;
5     case 2:
6         break;
7     default:
8 }
9

```

Figura 13: Estrutura básica do switch

Diferenciais

Apesar de praticamente tudo ser igual a linguagem C, o Javascript permite ao programador uma maior flexibilidade e poder, como por exemplo nos cases da figura 13, você pode colocar outros valores para ser comparado além de números ou um caractere apenas, você pode usar string e até objetos:

A imagem mostra um editor de código com um fundo escuro e uma barra de título com três pontos coloridos (vermelho, amarelo, verde). O código JavaScript está escrito em uma fonte monoespaçada com sintaxe destacada. O código define uma variável 'dado' com o valor 'Terça' e utiliza um switch statement para verificar o dia da semana. Os casos são 'Segunda-feira', 'Sábado', 'Domingo' e um caso 'default'. O caso 'Sábado' não possui um 'break', o que permite que o código do caso seguinte ('Domingo') seja executado também.

```
1 let dado = "Terça";
2 switch (dado) {
3   case "Segunda-feira":
4     console.log("É o primeiro dia da semana");
5     break;
6   case "Sábado":
7   case "Domingo":
8     console.log("É final de semana");
9     break;
10  default:
11    console.log("É um dia útil da semana");
12 }
13
```

Figura 14: Exemplo de uso

Perceba que no case “Sábado”, não existe um break, isso possibilita que ele execute o código que está no próximo case, é uma forma de fazer a mesma coisa para valores diferentes.

Outro diferencial é a capacidade de usar expressões nos cases caso você queira que algo aconteça dentro de um intervalo estabelecido, porém dentro do parâmetro do switch você deve colocar “true”, pois assim ele comparará qual expressão dos cases possui o valor verdadeiro, como por exemplo:

A imagem mostra um editor de código com um fundo escuro e uma barra de título com três pontos coloridos (vermelho, amarelo, verde). O código JavaScript utiliza um switch statement com o parâmetro 'true'. Os casos são baseados em expressões lógicas: 'numero <= 3', 'numero < 6' e 'numero <= 10'. Cada caso contém uma chamada de console.log e um 'break'. O caso 'default' também contém uma chamada de console.log.

```
1 let numero = 12;
2 switch (true) {
3   case numero <= 3:
4     console.log("Reprovado");
5     break;
6   case numero < 6:
7     console.log("Exame");
8     break;
9   case numero <= 10:
10    console.log("Aprovado");
11    break;
12  default:
13    console.log("Valor invalido");
14 }
```

Figura 15: Switch com uso de expressões lógicas

Sugestão de exercício

Colocar um valor entre 1 e 7 e dizer qual dia da semana corresponde ao valor comparado.

Resolução:

```
1 let nomeDia;
2 let numDia = 2;
3 switch (numDia) {
4   case 1:
5     nomeDia = "segunda-feira";
6     break;
7   case 2:
8     nomeDia = "terça-feira";
9     break;
10  case 3:
11    nomeDia = "quarta-feira";
12    break;
13  case 4:
14    nomeDia = "quinta-feira";
15    break;
16  case 5:
17    nomeDia = "sexta-feira";
18    break;
19  case 6:
20    nomeDia = "sábado";
21    break;
22  case 7:
23    nomeDia = "domingo";
24    break;
25  default:
26    nomeDia = "dia inválido";
27  }
28
29 // Exibe o nome do dia correspondente
30 console.log(`O dia correspondente é ${nomeDia}.`);
```

Figura 16: Resolução do exercício de switch

Funções

Introdução

Funções como já é conhecida, é um bloco de código reutilizável, sendo apenas necessário chamar seu nome que tudo o que estiver dentro dele será executado.

Declaração e uso

Na declaração de uma função basta escrever “function”, o nome da função e entre parênteses os parâmetros dessa função que será recebido da chamada externa e usados dentro do bloco de código.

Dentro das chaves você escreve o seu código e se quiser, você pode ou não usar o “return” para mandar para fora algum dado:

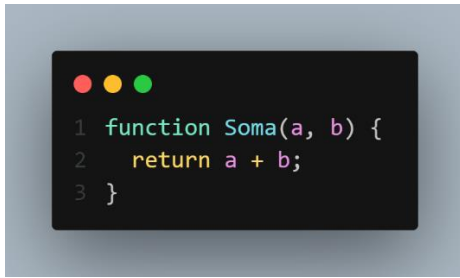


Figura 17: Estrutura básica de uma função

Para usar essa função, basta chamá-la pelo nome e passar seus valores por parâmetro:

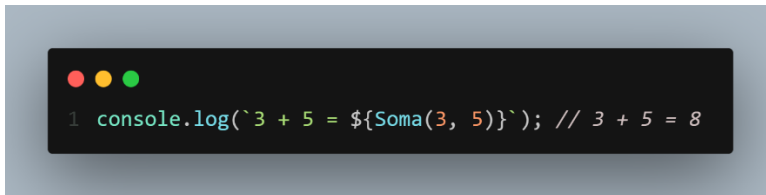


Figura 18: Chamada da função criada

Operador Spread

Introdução

Esse operador pode ser usado no parâmetro das funções quando você não sabe ou não tem um limite estabelecido da quantidade de variáveis a ser usada. Por exemplo, em vez de criar uma função para somar dois valores e depois criar outra função para somar três valores, basta usar o operador spread para fazer essas duas coisas ao mesmo tempo.

Sintaxe e uso

O seu uso é dado por reticências (...) seguido do nome do parâmetro, e quando você fizer isso, esse parâmetro atuará como um array com todos os valores recebidos da chamada externa, e dentro da função você poderá manipular esse array:



Figura 19: Uso de operador spread no parâmetro

Ao fazer isso, perceba que agora na chamada externa podemos passar uma quantidade indefinida de valores por parâmetro e tudo funcionará perfeitamente:

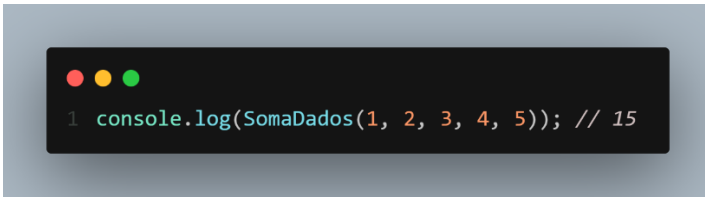


Figura 20: Chamada da função passando 5 valores

Arrow Functions

Introdução

Esse tipo de função é uma função anônima, isso quer dizer que ela não possui nome, deve ser confuso de imaginar como chamar uma função se ela não tem o nome para ser chamada, porém, essa maneira de usar funções é a mais utilizada entre os programadores de Javascript pois, de maneira mais básica, deixa o código mais elegante e conciso além de ser o principal agente em high-order functions.

Sintaxe e uso

A arrow function como o próprio nome diz, possui um formato de flecha, possui os parênteses para receber os parâmetros e os colchetes para escrever o código:



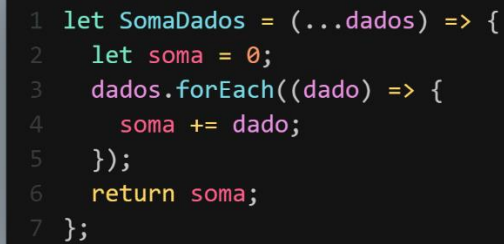
Figura 21: Estrutura básica da arrow function

Para facilitar a compreensão do uso vamos alterar o código da figura 19 usando arrow function:



Figura 22: Uso da arrow function

Ou até mesmo:



```

1 let SomaDados = (...dados) => {
2   let soma = 0;
3   dados.forEach((dado) => {
4     soma += dado;
5   });
6   return soma;
7 };

```

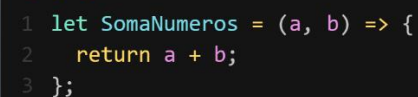
Figura 23: Outro possível uso da arrow function

Ambos funcionarão da mesma forma. Resumidamente, a palavra reservada “function” foi substituída pela flecha “=>”.

Método Reduce

Esse método é uma boa forma de demonstrar a aplicação do mesmo código de soma da função anterior, porém usando apenas uma linha. Para isso, conheceremos também um novo conceito de “return” da arrow function.

Normalmente para retornarmos um valor para fora da função usamos o conhecido “return”, porém, numa arrow function isso não é necessário quando você tem um código simples o suficiente para não ocupar duas linhas, quando isso acontecer, basta você tirar as chaves da arrow function que ela entendera que o próximo conteúdo deve ser retornado, para facilitar a visualização, vamos comparar:



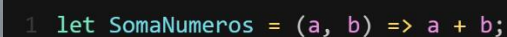
```

1 let SomaNumeros = (a, b) => {
2   return a + b;
3 };

```

Figura 24: Arrow function de soma básica

Perceba o uso das chaves e da palavra reservada “return”, três linhas de código para somar dois números, isso pode ser resolvido ao remover as chaves e a palavra return:



```

1 let SomaNumeros = (a, b) => a + b;

```

Figura 25: Arrow function com retorno implícito

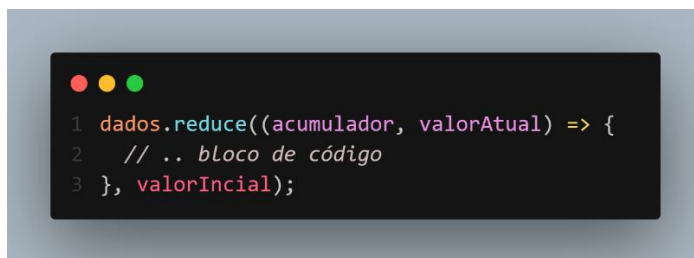
Funcionará exatamente da mesma maneira, dessa maneira o Javascript entende de forma implícita o retorno do resultado.

Obs.: Caso a função tenha mais linhas de código, então é necessário voltar a fazer o uso das chaves pois mais de uma linha já se tornou um bloco de código.

Agora que isso foi entendido, vamos voltar ao assunto principal que é o uso do método `reduce` para soma de quantidade indefinida de números.

O método `reduce` nada mais é do que um acumulador, ele reduz uma variedade de valores em um único lugar, para fins didáticos usaremos isso para soma que é o mais simples, onde ele pegará vários valores e acumulará dentro de apenas uma variável. O `reduce` recebe dois parâmetros, o primeiro que é a *call-back function* e o segundo que é o valor inicial desse acúmulo, seja 0, ou um array [], ou um objeto {}.

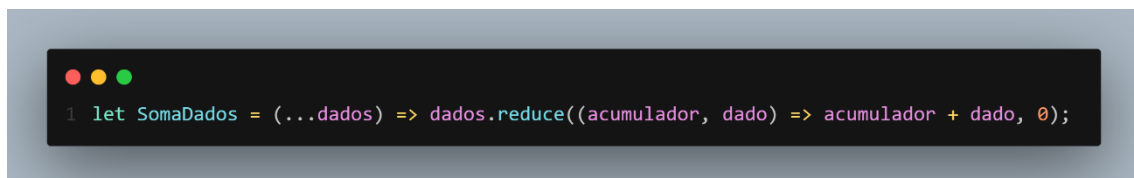
Uma *call-back function* nada mais é do que uma função anônima que faz o método funcionar, onde é especificado como aquele método deve se comportar, e a função *call-back* do `reduce` recebe 4 coisas por parâmetro, mas vamos focar em duas, a primeira que é o acumulador onde o resultado será acumulado, e o segundo que é o valor atual do array a ser “reduzido”:



```
1 dados.reduce((acumulador, valorAtual) => {  
2   // .. bloco de código  
3 }, valorIncial);
```

Figura 26: Estrutura básica do método `reduce`

Para fazer a soma de vários valores, vamos fazer o método acumular todos os valores do array na variável acumuladora, e retornarmos esse resultado para a chamada externa, porém como você já sabe, implementações muito simples não precisam de `return`, resultado num código de apenas uma linha:



```
1 let SomaDados = (...dados) => dados.reduce((acumulador, dado) => acumulador + dado, 0);
```

Figura 27: Soma de vários valores em apenas uma linha

Veja que a função `SomaDados` é uma *arrow function* que retorna, de forma implícita, o valor acumulado dos dados recebidos, e a *arrow function* dentro do `reduce` acumula cada valor desse array e, por fim, retorna o resultado também de forma implícita, e no final o valor inicial do acumulador, 0.

Sugestão de exercício

Criar uma função que retorna um array contendo os valores de 1 até 10, outra que retorna um array com todos os valores dobrados, outra que retorna um array apenas com os números pares e outra que soma todos os valores do array.

Resolução:

```
1 // Função para criar um array de números de 1 a 10
2 function criarArray() {
3   let array = [];
4   for (let i = 1; i <= 10; i++) {
5     array.push(i);
6   }
7   return array;
8 }
9
10 // Função para dobrar os números de um array
11 function dobrarNumeros(array) {
12   return array.map(num => num * 2);
13 }
14
15 // Função para filtrar números pares de um array
16 function filtrarPares(array) {
17   return array.filter(num => num % 2 === 0);
18 }
19
20 // Função para somar todos os elementos de um array
21 function somarArray(array) {
22   return array.reduce((acc, num) => acc + num, 0);
23 }
24
25 // Criar um array de 1 a 10
26 let numeros = criarArray();
27 console.log("Array original:", numeros);
28
29 // Dobrar os números do array
30 let numerosDobrados = dobrarNumeros(numeros);
31 console.log("Números dobrados:", numerosDobrados);
32
33 // Filtrar os números pares do array dobrado
34 let numerosPares = filtrarPares(numerosDobrados);
35 console.log("Números pares dobrados:", numerosPares);
36
37 // Somar os números pares filtrados
38 let soma = somarArray(numerosPares);
39 console.log("Soma dos números pares dobrados:", soma);
40
```

Figura 28: Resolução do exercício de funções

Detalhe importante: perceba que existem alguns métodos que não foram mostrados aqui como map e filter, nessa hora uma boa sugestão é falar que “Grande parte do que você precisa não vai ser ensinado pra você”, que a pessoa deve aprender a pesquisar e procurar por si só, seja pesquisando no Google, vendo a documentação etc. Isso ajudará aos estudantes perceberem como o mundo real é, e caso tentem fazer na mão algo que um

método já existente faz, talvez aprendam que as vezes é necessário pesquisar primeiro se já existe um método que faz o que ele deseja antes de tentar recriar a roda.

Exercício Final

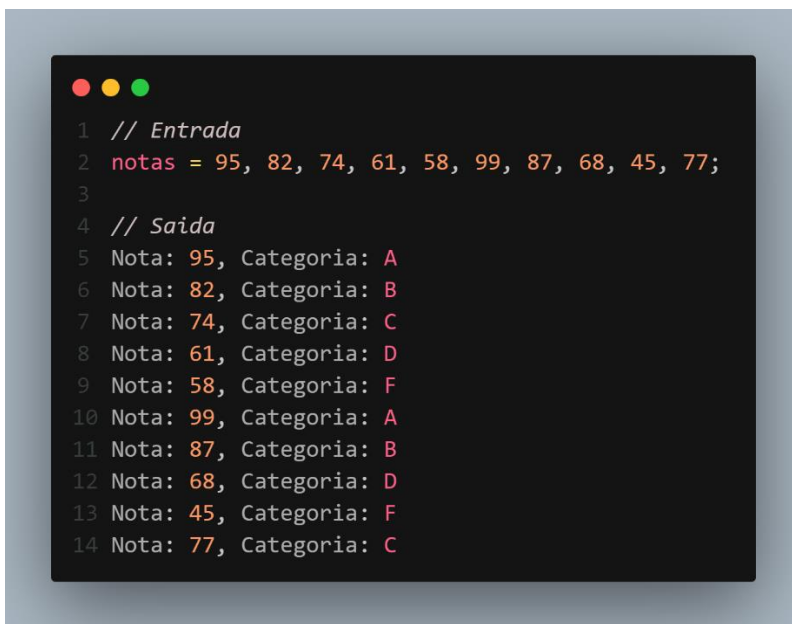
Esse é um exercício simples envolvendo array, switch e funções.

Deve ser criado um array com notas de um número indefinido de alunos, essas notas irão variar entre 0 e 100, o código deve verificar cada uma das notas e classificar por categorias e em seguida mostrar no console a nota e sua categoria respectiva.

As categorias são:

		Categorias
Notas	Entre 100 e 90	A
	Entre 90 e 80	B
	Entre 80 e 70	C
	Entre 70 e 60	D
	Entre 60 e 0	F

Exemplo de notas e saída:

A screenshot of a code editor with a dark background and light-colored text. The code is in JavaScript and demonstrates an array of grades and a loop that classifies each grade into a category (A, B, C, D, F) based on a switch statement. The output is printed to the console for each grade.

```
1 // Entrada
2 notas = 95, 82, 74, 61, 58, 99, 87, 68, 45, 77;
3
4 // Saída
5 Nota: 95, Categoria: A
6 Nota: 82, Categoria: B
7 Nota: 74, Categoria: C
8 Nota: 61, Categoria: D
9 Nota: 58, Categoria: F
10 Nota: 99, Categoria: A
11 Nota: 87, Categoria: B
12 Nota: 68, Categoria: D
13 Nota: 45, Categoria: F
14 Nota: 77, Categoria: C
```

Figura 29: Entrada e saída de dados

Resolução:

```
1 // Array de notas dos alunos
2 let notas = [95, 82, 74, 61, 58, 99, 87, 68, 45, 77];
3
4 // Função para classificar uma nota
5 function classificarNota(nota) {
6   let categoria;
7
8   switch (true) {
9     case nota >= 90 && nota <= 100:
10      categoria = "A";
11      break;
12     case nota >= 80 && nota < 90:
13      categoria = "B";
14      break;
15     case nota >= 70 && nota < 80:
16      categoria = "C";
17      break;
18     case nota >= 60 && nota < 70:
19      categoria = "D";
20      break;
21     case nota >= 0 && nota < 60:
22      categoria = "F";
23      break;
24     default:
25      categoria = "Nota inválida";
26   }
27
28   return categoria;
29 }
30
31 // Função para avaliar todas as notas de um array
32 function avaliarNotas(notas) {
33   notas.forEach((nota) => {
34     let categoria = classificarNota(nota);
35     console.log(`Nota: ${nota}, Categoria: ${categoria}`);
36   });
37 }
38
39 // Avaliando as notas
40 avaliarNotas(notas);
```

Figura 30: Resolução do exercício final

Exemplos Trabalhados:



```
1 function somadados(...dados){
2   let soma =0;
3   dados.forEach( function(dado){
4     soma+=dado;
5   });
6   return soma;
7 }
8 console.log(somadados(1, 2, 3 ,4));
```



```
1 function soma(a,b)
2 {
3   return a*b
4 }
5 console.log("3+5 = ", soma(3, 5))
```



```
1 const soma=(v1,v2)=>v1+v2
2 console.log(soma(10,4))
3
4 const nome=n=>n
5 console.log("Ana")
6
7 const add=n=>n+10
8 console.log(add(10))
9
```



```
1 let somadados= (...dados)=>dados.reduce((somatoria, dado)=> somatoria+dado, 0);
2
3 console.log(somadados(1, 2, 3 ,4 ,5));
```

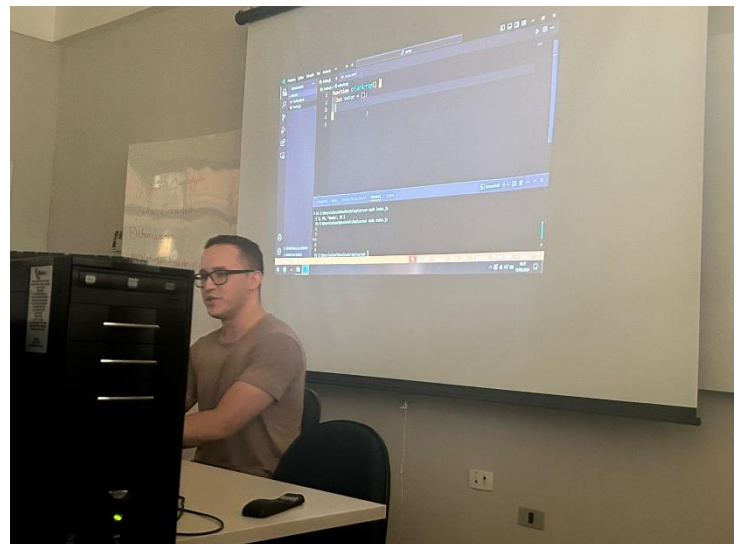
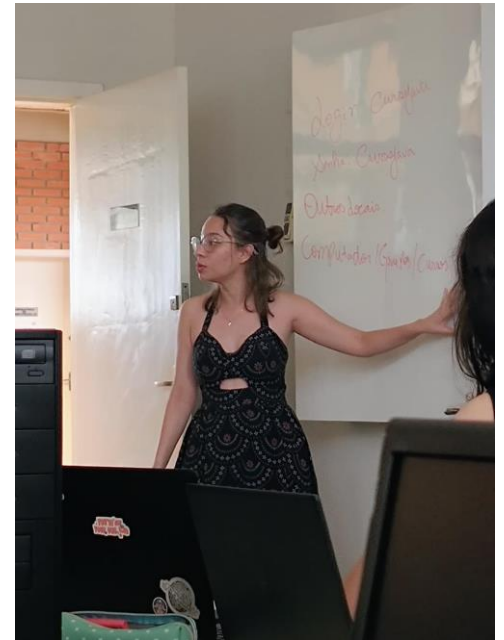


```
1 const soma=(v1,v2)=>v1+v2
2 console.log(soma(10,4))
3
4 const nome=n=>n
5 console.log("Ana")
6
7 const add=n=>n+10
8 console.log(add(10))
9
```



```
1 //function somadados(...dados){
2   //let soma =0;
3   //dados.forEach( function(dado){
4     //soma+=dado;
5
6   //});
7   //return soma;
8 //}
9 //console.log(somadados(1, 2, 3 ,4 ,5));
10
11 let somadados=(...dados)=> {
12   let soma=0;
13   dados.forEach((dado)=>{
14     soma +=dado;
15
16   });
17   return soma;
18 }
19 console.log(somadados(1, 2, 3 ,4 ,5));
20
```

Fotos:





ABAIXO ARQUIVOS RELACIONADOS A AULA 3. APLICADA DIA 20/09/2024

Em ordem os arquivos previamente planejados.

Em seguida o exercício apresentado e sua resolução, por último as fotos referentes ao dia.

Sumário

HTML.....	2
Introdução	2
Estrutura básica	2
Tags principais	3
Títulos	3
Parágrafos	4
Listas	4
Links	5
Forms.....	6
Exercício	7
DOM	8
Introdução	8
Inserindo o arquivo no HTML	9
Conceitos chave	10
Métodos de seleção.....	10
Manipulando o DOM.....	12
Alterando o texto	12
Alterando os atributos.....	13
Adicionando e removendo elementos.....	13
Exercício	15
Eventos.....	16
Adicionando eventos	16
Exercício	17
Extra.....	18
Entrada de forma simples	18
Saída de forma simples	18

HTML

Introdução

HTML é uma linguagem usada para a criação de páginas web, se formos separar toda a estrutura web de forma metafórica, o HTML é como o esqueleto do corpo humano, a estrutura básica como os ossos, o CSS será responsável pela aparência, então seria como a cor de pele, altura da pessoa, cor dos olhos entre outros detalhes, e o Javascript o comportamento do corpo humano, o que os músculos fazem, como o coração bate e entre outros mecanismos.

Diferente do Javascript, o HTML **não** é uma linguagem de programação, isso porque nenhuma implementação lógica é usada, sendo apenas baseadas em tags que são elementos HTML.

Estrutura básica

O HTML por padrão possui uma estrutura que sempre será utilizada em seus projetos.

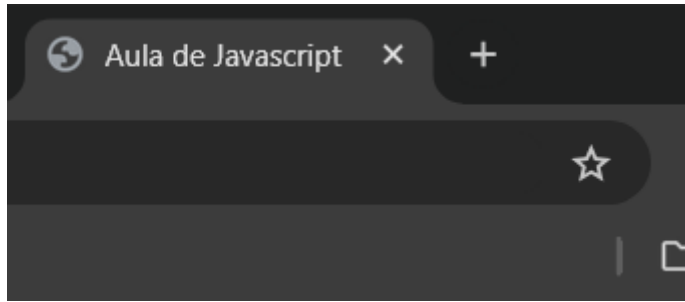


Figura 1: Estrutura básica HTML

Vamos entender cada parte:

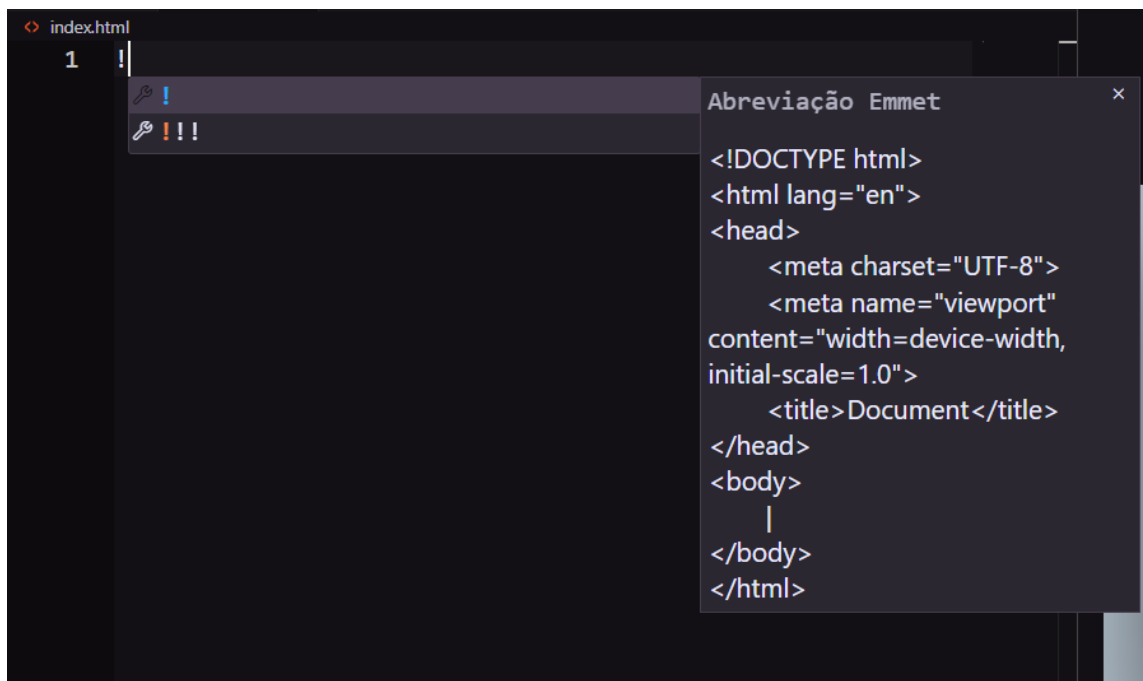
1. `<!DOCTYPE html>`: Serve para comunicar ao navegador, seja ele o Chrome, Firefox, etc. Que o documento que está sendo criado é o HTML5.
2. `<html lang="pt-BR">`: É a tag principal que vai englobar todo o conteúdo da página que você estiver criando, e nesse exemplo, essa tag possui um atributo extra, `lang:"pt-BR"` que informa para o navegador que o idioma a ser usado é o português Brasileiro.
3. `<head>`: Aqui ficará todos os metadados, que são informações implícitas passadas ao navegador, e dentro dessa tag `<head>` existe as informações básicas como o `<meta charset="UTF-8">` que é uma padronização de acentuações, assim a página entende as acentuações específicas da linguagem brasileira. O segundo metadado é o `<meta name="viewport" ...>`, aqui ele está deixando a página adaptável para cada tipo de tela diferente, ou seja, o tamanho da página irá se adaptar se você estiver usando um celular ou se estiver no computador. E outra

tag dentro do head é o <title> que é o título da página que aparece visivelmente ao usuário:



4. <body>: Por fim, dentro dessa tag ficará todo o conteúdo que será exibido na página, ou seja, títulos, textos, imagens etc.

Apesar de ser diversas informações, não é necessário decorar nenhuma delas, por padrão a IDE monta automaticamente todas essas informações ao clicar ponto de exclamação (!) no arquivo HTML vazio:



E ao clicar enter, todas as informações serão geradas automaticamente.

Tags principais

Em HTML, uma tag é formada pelos sinais < e > e entre eles o nome da tag, como por exemplo <tag>, e para fechar essa tag, você usa uma barra (/) antes do nome da tag. Por exemplo “<tag> Conteúdo </tag>”, o que está sendo feito aqui é a abertura de uma tag, a inserção do conteúdo entre elas e por fim fechando a tag que foi aberta.

Títulos

Em HTML, os títulos são representados de <h1> até <h6>, sendo que os números simbolizam o nível de importância desse título, quanto maior for o número, menor a significância, isso quer dizer que a tag h6 vai ser menor que a tag h5, a tag h5 vai ser menor que a h4 e por assim vai, como por exemplo:

```

1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Aula de Javascript</title>
7    </head>
8    <body>
9      <h1>Título Principal</h1>
10     <h2>Subtítulo</h2>
11     <h3>Subtítulo menor</h3>
12   </body>
13 </html>

```

Título Principal

Subtítulo

Subtítulo menor

Parágrafos

Os parágrafos é onde ficará os seus textos e são representado pela tag <p>, dentro dessa tag você ainda pode inserir outras para dar uma ênfase caso desejado, como que deixa as palavras em negrito, e o que deixa em itálico:

```

1  <!DOCTYPE html>
2  <html lang="en">
3  > <head> ...
7    </head>
8    <body>
9      <p>
10        Este é um <strong>parágrafo
11        importante</strong> <br />com
12        algum
13        <em>texto em itálico</em>.
14      </p>
15    </body>
16  </html>

```

Este é um **parágrafo importante**
com algum *texto em itálico*.

Perceba que também foi utilizado a tag
, que é responsável por pular uma linha caso você deseje.

Listas

Existem dois tipos de listas em HTML, as não ordenadas e as ordenadas, as não ordenadas são listas que não tem contadores, ou seja, serão apenas marcados com pontos em cada item da lista, e as ordenadas terá uma enumeração indo de 1 até a quantidade de itens na lista.

A lista não ordenada é representada por que significa “unordered list” e dentro dessa tag existirá as tags que significa “list item” que é o item da lista, por exemplo:

```

1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 > <head> ...
7 </head>
8 <body>
9 <h3>Lista não ordenada</h3>
10 <ul>
11 <li>Item 1</li>
12 <li>Item 2</li>
13 <li>Item 3</li>
14 </ul>
15 </body>
16 </html>

```

Lista não ordenada

- Item 1
- Item 2
- Item 3

E lista ordenada funciona exatamente da mesma forma, a diferença é que você vai trocar a tag pela :

```

1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 > <head> ...
7 </head>
8 <body>
9 <h3>Lista ordenada</h3>
10 <ol>
11 <li>Item 1</li>
12 <li>Item 2</li>
13 <li>Item 3</li>
14 </ol>
15 </body>
16 </html>

```

Lista ordenada

1. Item 1
2. Item 2
3. Item 3

Links

Usamos a tag <a> para linkar o que quisermos, como por exemplo um site na internet ou até outra página HTML que você criou, isso é útil pois caso você queira criar várias páginas e permitir que o usuário navegue entre elas, você vai ter que utilizar essa tag para endereçar o usuário para a página desejada. Porém a tag <a> sozinha não faz nada, é necessário utilizar um atributo dela chamado "href" e passar para esse atributo o link ou o caminho da página que você quer acessar:

```

1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 > <head> ...
7 </head>
8 <body>
9 <a href="https://www.google.com"
10 target="_blank">Acesse o Google</a>
11 </body>
12 </html>

```

[Acesse o Google](https://www.google.com)

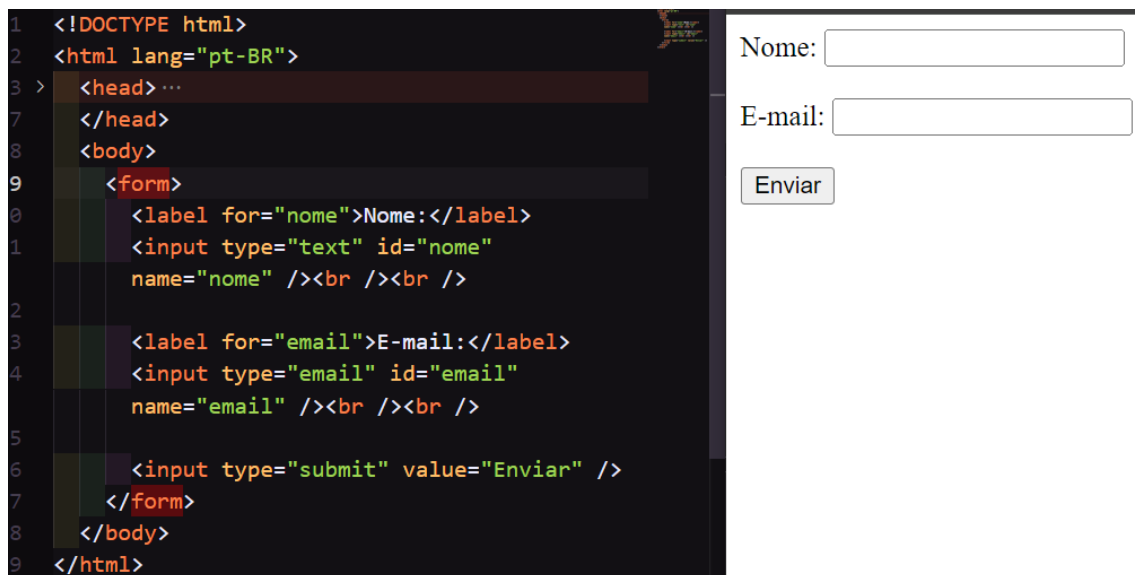
Perceba que um texto "clicável" foi gerado, quando o usuário clicar nesse texto ele será direcionado à página do google. Como extra, o atributo "target" recebendo "_blank" faz

que quando o usuário clicar no link, o link seja aberto numa nova aba e não na aba onde o link está.

Forms

Forms nada mais são do que formulários, como aqueles que você preenche ao se cadastrar em algum site ou quando você tenta logar em algum lugar, como por exemplo o seu sistema acadêmico, aquilo é um form.

O objetivo do form é coletar os dados do usuário através de inputs (entradas), representado pela tag `<input>`, essa tag ficará dentro do form. E essa tag `<input>` possui um atributo chamado “type” onde você consegue escolher que tipo de input você deseja, ou seja, se você quer um input do tipo email, então você escolhe “email” (ele fará a verificação automática se o formato do texto inserido equivale a um email), se você quer apenas capturar apenas números, você diz que é do tipo “number” (só aceitará números pelo usuário, qualquer outra coisa será ignorada). Aqui está um exemplo:



The image shows a side-by-side comparison of HTML code and its rendered output. On the left, a code editor displays the following HTML code:

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 > <head> ...
7 </head>
8 <body>
9   <form>
10     <label for="nome">Nome:</label>
11     <input type="text" id="nome"
12       name="nome" /><br /><br />
13
14     <label for="email">E-mail:</label>
15     <input type="email" id="email"
16       name="email" /><br /><br />
17
18     <input type="submit" value="Enviar" />
19   </form>
20 </body>
21 </html>
```

On the right, the rendered form is shown. It consists of two text input fields. The first field is preceded by the label "Nome:" and the second by "E-mail:". Below these fields is a button labeled "Enviar".

Perceba que existem outra tag chamada `<label>`, ela representa um texto associado a um input, isso serve para caso o usuário clique nesse label, a caixa do input vai ser selecionada automaticamente. E existe o tipo “submit” usando no final do form, que serve para enviar o formulário para algum lugar desejado (conteúdo relacionado a servidores).

Outro detalhe muito importante para ressaltar são os atributos usados nas tags de input, o atributo “id”, esse atributo significa “identificador” que é **único** para cada elemento HTML, ou seja, se você escolher um id para um elemento, você não pode usar o mesmo id para outro elemento em todo arquivo HTML, é como um CPF para nós, nunca vai existir no Brasil outra pessoa com o mesmo CPF que você.

O uso do ID vai ser extremamente utilizado na manipulação do DOM para acessarmos um elemento específico da página, é como dar um nome para um elemento a qual poderemos identificar e usar depois.

Em todos os elementos também é possível usar o atributo “class” que é usado para estilizar um elemento em CSS, o nome da class de um elemento pode ser igual ao outro,

isso facilita para que quando você estilize um elemento, outros com a mesma class também sofra alteração.

Exercício

Replique a página HTML seguinte:

Notícias

Saiba dos **principais tópicos** abordados na tecnologia

1. Inteligência artificial
2. Compartilhamento de dados
3. Automação de processos

[Clique aqui](#) para saber mais

Matenha-se atualizado

Nome: Email:

Resolução:

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Aula de Javascript</title>
7     <script src="./index.js" defer></script>
8   </head>
9   <body>
10    <h1>Notícias</h1>
11    <p>Saiba dos <strong>principais tópicos</strong> abordados na tecnologia</p>
12    <ol>
13      <li>Inteligência artificial</li>
14      <li>Compartilhamento de dados</li>
15      <li>Automação de processos</li>
16    </ol>
17    <p><a href="www.google.com">Clique aqui</a> para saber mais</p>
18    <h3>Mantenha-se atualizado</h3>
19    <form action="">
20      <label for="name">Nome: </label>
21      <input type="text" />
22      <label for="email">Email: </label>
23      <input type="email" />
24      <input type="submit" value="Enviar" />
25    </form>
26  </body>
27 </html>
```

DOM

Introdução

O DOM significa Document Object Model (Modelo de objeto de documento) e é uma interface que permite a programação de documentos HTML. O DOM vê a estrutura HTML como uma árvore de objetos, então por exemplo, essa é a nossa visão do HTML ao lado de como o DOM vê:



Perceba que o DOM trata tudo como uma hierarquia, onde a tag html possui as tags head e body, e na tag body existe as tags h1 e p, e dentro de p existe a tag strong.

O DOM é usado para manipularmos as páginas web de forma dinâmica, isso quer dizer que o usuário não precisa recarregar a página para que uma alteração seja feita, como ao clicar o botão e mudar um texto instantaneamente, tudo é alterado em tempo real.

Inserindo o arquivo no HTML

Existe duas formas de você inserir o seu código Javascript no seu documento HTML, um deles é inserindo a tag `<script>` no final da tag `<body>`, como por exemplo:

```
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Aula de Javascript</title>
  </head>
  <body>
    <h1 id="titulo">Título</h1>

    <!-- Conteúdo -->

    <script src="index.js"></script>
  </body>
</html>
```

E a outra forma é inserindo dentro da tag `<head>`, porém caso você escolha essa segunda forma, você deve usar o atributo `defer` da tag `<script>`, isso diz ao navegador “Espere o documento HTML carregar e depois carregue o Javascript”, isso é porque caso o Javascript seja carregado antes do HTML, ele pode tentar procurar elementos HTML que ainda nem foram gerados pelo HTML:


```

<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
  <title>Aula de Javascript</title>
    <script src="./index.js" defer></script>
  </head>
  <body>
    <h1 id="titulo">Título</h1>

    <!-- Conteúdo -->

  </body>
</html>

```

Conceitos chave

O primeiro passo de toda manipulação DOM é pegar do HTML um elemento que você deseja, aqui se torna necessário que o elemento HTML tenha um id como dito anteriormente.

Manipular o DOM segue um seguinte padrão:

document.método: O document diz que você está se comunicando com o HTML e o método é o que você quer fazer.

Métodos de seleção

Pelo ID

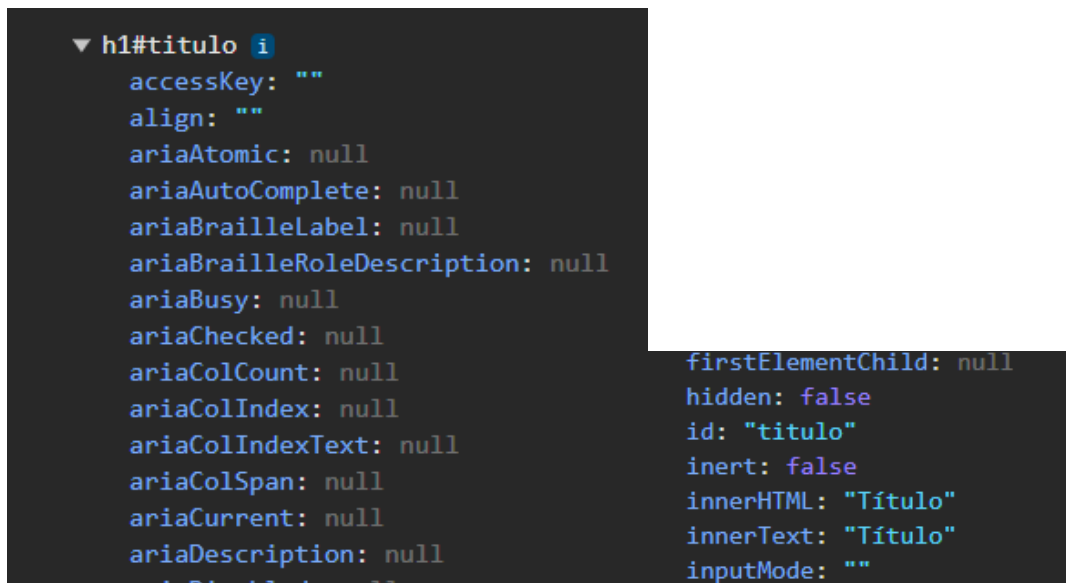
Então caso você queira pegar um elemento do HTML pelo id, você terá o método “document.getElementById(‘id’)”, perceba como os métodos são autoexplicativos, nesse exemplo, ele pega o elemento pelo id:

```

<title>Aula de Javascript</t
<script src="./index.js" def
</head>
<body>
  <h1 id="titulo">Título</h1>
</body>
</html>
6
7
8
9
10 const titulo = document.getElementById("titulo");
11
12
13

```

Perceba ao pegar o elemento pelo ID, ele deve ser guardado em algum lugar, e esse lugar vai ser uma variável Javascript que sempre será constante, por isso o “const”, isso porque uma vez que você pegar o elemento HTML, você não vai fazer a mesma variável receber outro elemento. E caso você dê um console.log(titulo), isso aparecerá no seu navegador:

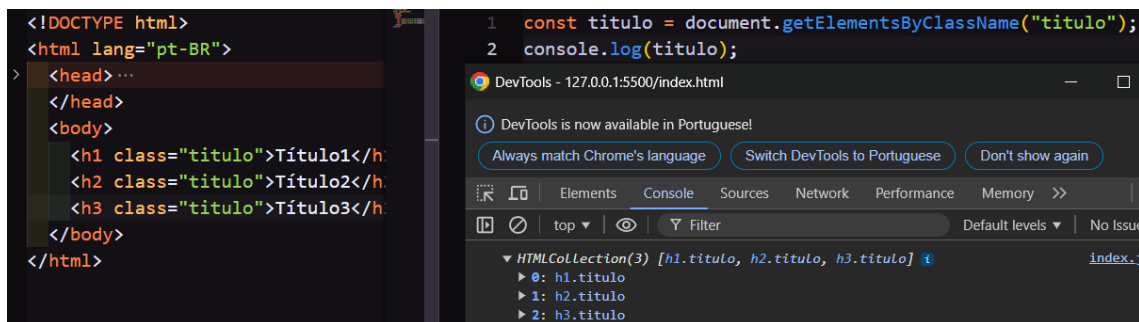


O elemento será mostrado com todos os atributos possíveis existentes, e um deles, como mostrado na imagem direita, é o ID o texto dentro do título. Isso significa que todos esses atributos podem ser alterados usando Javascript.

Pela classe

Outra forma de selecionar elementos é pela classe utilizando

“document.getElementsByClassName”, a diferença é que enquanto o ID pode ser apenas um, a classe como dito anteriormente pode ser usado por vários elementos, por causa disso caso o Javascript perceba que existe mais de um elemento usando a mesma classe, ele coloca todos esses elementos dentro de um vetor onde você pode manipular cada um individualmente:

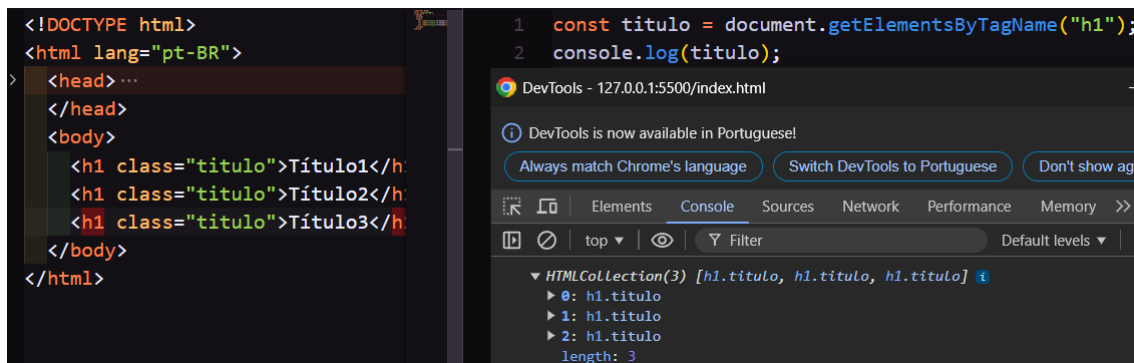


Detalhe: Caso você queira iterar sobre esses elementos, o método forEach não funcionará, você precisa usar o clássico for.

Pela tag

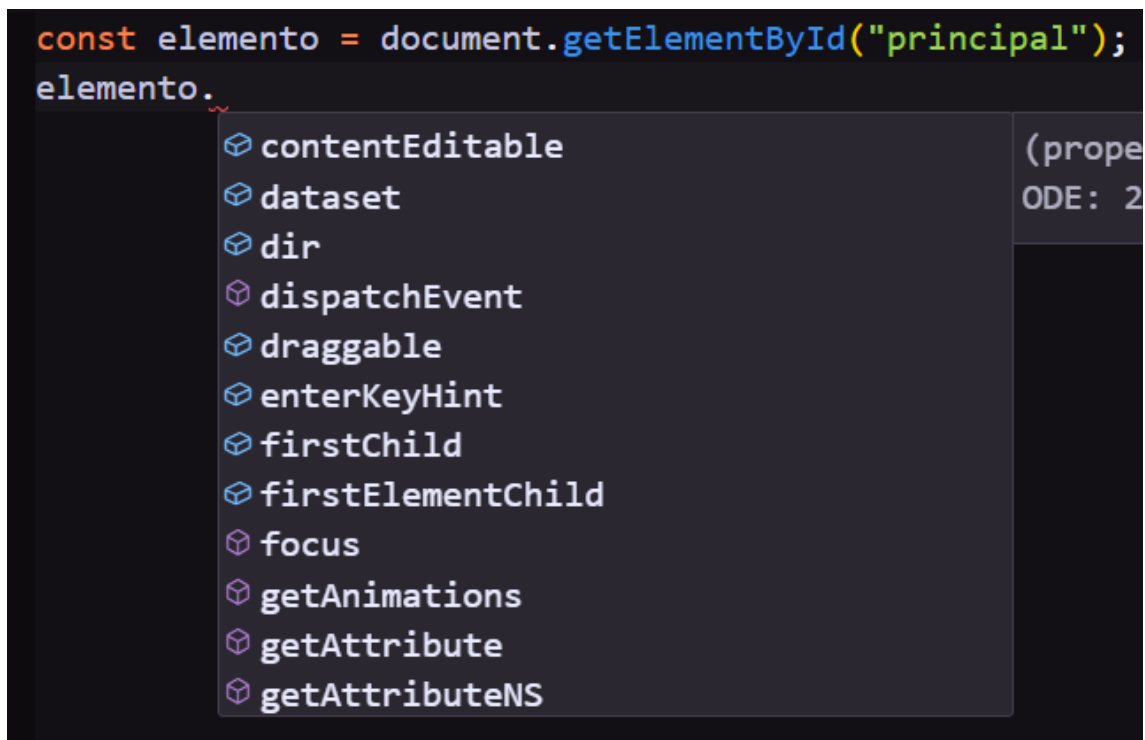
Caso você queira pegar pela tag, isso também é possível utilizando

“document.getElementsByTagName”, da mesma forma da classe, ele vai pegar todas as tags que encontrar no HTML e colocar dentro de um vetor:



Manipulando o DOM

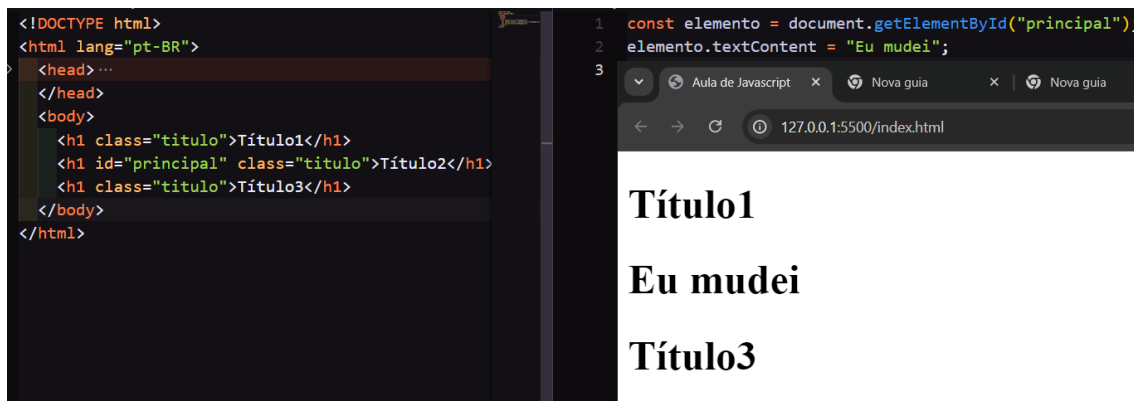
Uma vez pego o elemento HTML, você está livre para fazer o que quiser com ele, mudar o texto, mudar seus atributos, mudar o estilo, tudo pelo Javascript, clicando ponto (.) na variável onde está seu elemento, a IDE exibirá diversos usos.:



Vamos mostrar alguns exemplos:

Alterando o texto

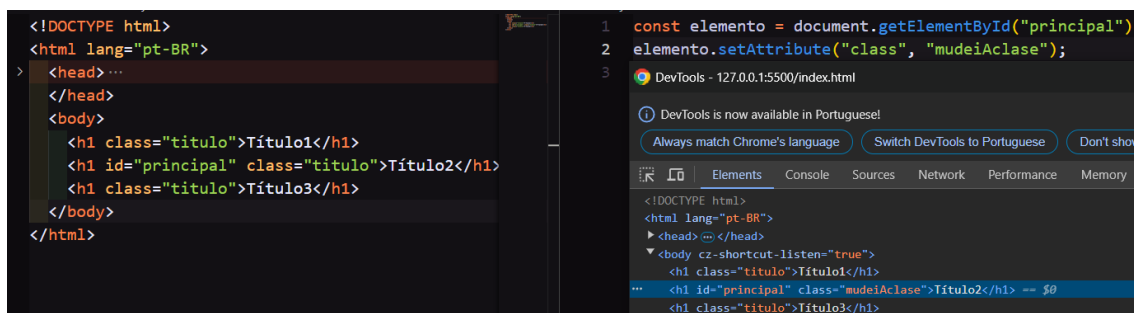
Ao guardar o elemento HTML numa variável, ao usar o método “`textContent`”, você consegue alterar o conteúdo do texto. É importante notar como os métodos possuem nomes de fácil compreensão e intuitivos:



Veja que o elemento com o id “principal” foi pego do HTML e então, com `textContent` seu valor que era “Título2” passou a ser “Eu mudei”.

Alterando os atributos

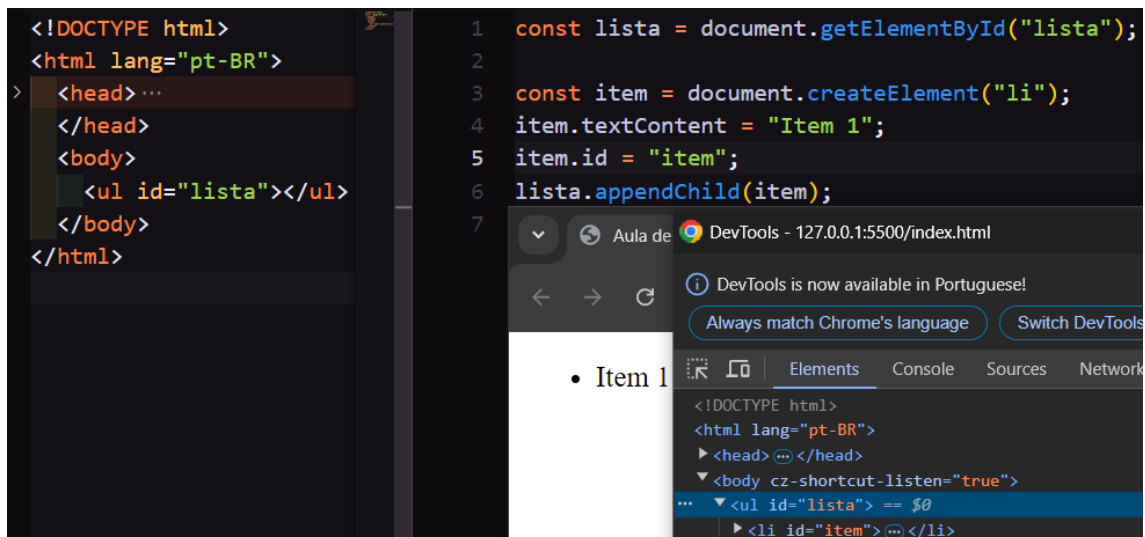
No Javascript você consegue alterar os atributos das tags que você pegar, como a tag `class`, `id`, `src`, `href` e até mesmo criar outros atributos. Isso é feito com o método “`setAttribute`” e como o próprio nome diz, ele “seta” um atributo, ele define um atributo, e em seu parâmetro você precisa passar duas coisas, o nome do atributo e o valor desse atributo. Como por exemplo:



Veja que a classe original do Titulo2 era “titulo” e após o `setAttribute` ele foi alterado para “mudeiAclase”. Caso você queira apagar um atributo, você apenas pode usar o `removeAttribute`(“atributo”) e o atributo com o nome que você passar será apagado.

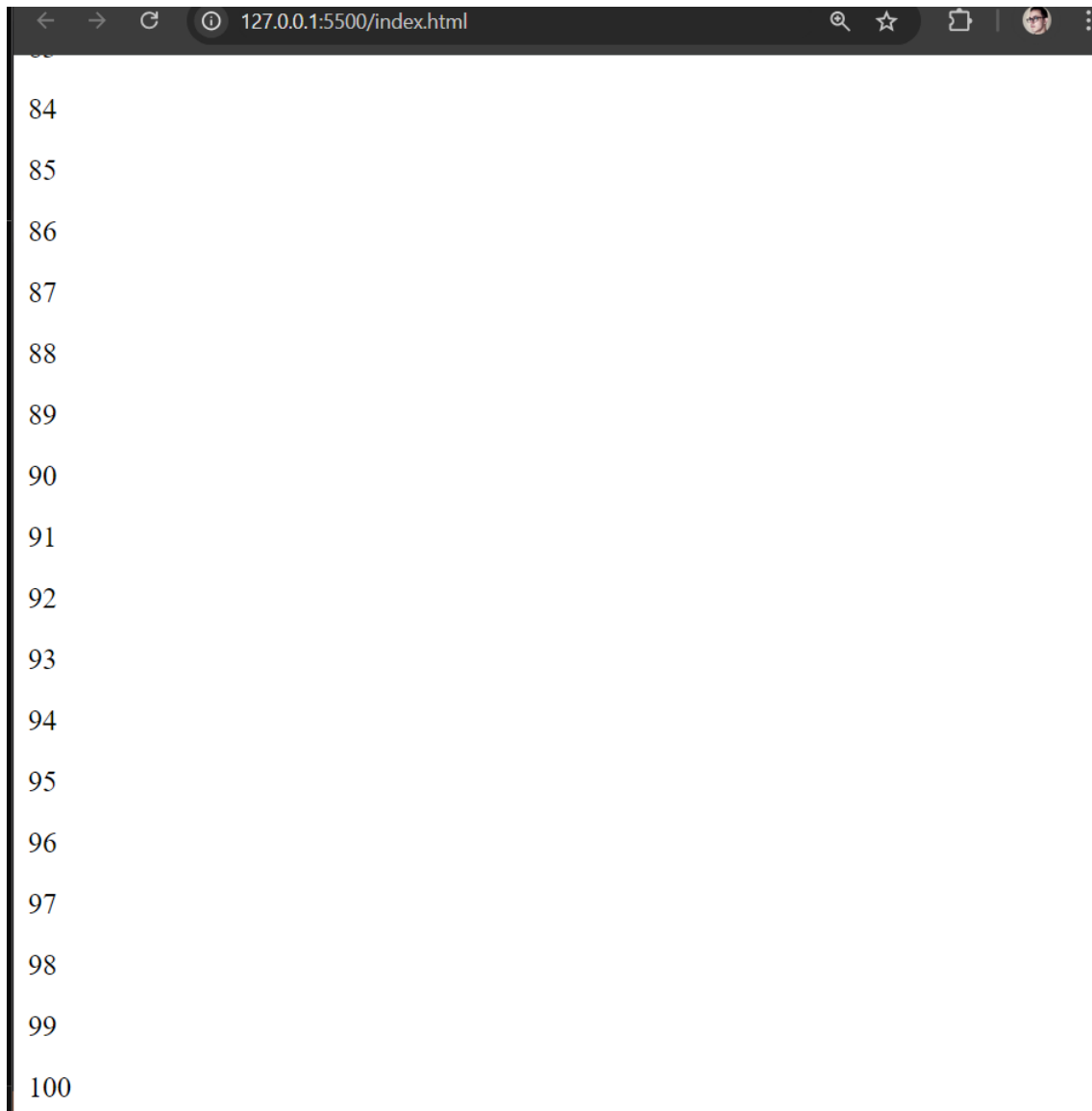
Adicionando e removendo elementos

Também é possível inserir e remover elementos diretamente pelo Javascript com dois métodos, o primeiro é o “`createElement`” que cria um elemento da tag que você passar, e o método “`appendChild`” que você usa no elemento pai. Como vimos anteriormente, o DOM vê os elementos como uma árvore de hierarquia, então você precisa de um elemento pai para que seja inserido um elemento filho. Então vamos ver um exemplo:



Exercício

Faça um código em Javascript que insira os números de 1 a 100 na página HTML, como por exemplo:



Resolução:

```
for (let i = 1; i <= 100; i++) {  
  const elemento = document.createElement("p");  
  elemento.textContent = i;  
  document.body.appendChild(elemento);  
}
```

Perceba como body ser uma tag padrão do HTML, não foi necessário “pegar” ela pela tag ou dar um id.

Eventos

Eventos são as ações que ocorrem na página, como por exemplo um clique, o ato de enviar um form, o ato de digitar num campo de texto. O Javascript tem o poder de “escutar” esses eventos e realizar alguma ação que você desejar.

Adicionando eventos

Todo elemento pego do HTML possui um método chamado “addEventListener” que, como o próprio nome diz, escuta um evento que acontecerá nesse elemento, existindo alguns principais como:

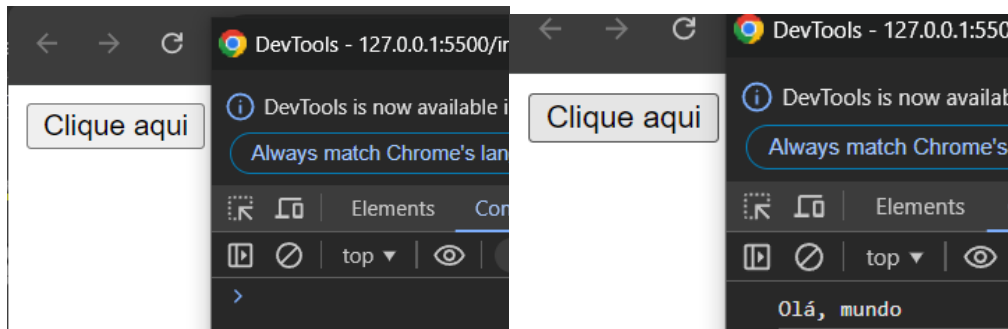
- “click”: Algo acontece quando o elemento for clicado.
- “mouseover”: Algo acontece quando o cursor do mouse passa por cima do elemento.
- “submit”: Disparado quando o usuário envia um formulário.
- “keydown” e “keyup”: Um evento acontece quando a tecla é pressionada ou liberada.

O método “addEventListener” possui dois parâmetros, o primeiro é o nome do evento como alguns citados anteriormente, esse nome estará entre aspas. E o segundo é uma função anônima, que é a função que vai dizer o que deve acontecer quando o evento escolhido for disparado.

Vamos criar um exemplo que escreva “Olá, mundo” ao clicar num botão na página:

```
<!DOCTYPE html>  
<html lang="pt-BR">  
> <head> ...  
  </head>  
  <body>  
    <button id="botao">Clique aqui</button>  
  </body>  
</html>  
  
1 const botao = document.getElementById  
  ("botao");  
2 botao.addEventListener("click", function () {  
3   console.log("Olá, mundo");  
4 });  
5
```

Criamos um botão no HTML, pegamos ele no Javascript pelo seu ID e então adicionamos um evento dele de “click” que será disparado quando o usuário clicar no botão. Dentro da função anônima dizemos para exibir no console o texto “Olá, mundo” quando o botão for clicado:



Após clicar, vemos que o evento foi disparado.

Exercício

Crie um projeto “Lista de desejos”. O site deve ter um título “Lista de desejos” e um input do tipo texto para o usuário inserir o que quiser, ao clicar num botão, o conteúdo que ele digitou deve ser colocado como o item de uma lista (pode ser ordenada ou desordenada).

Resolução:



Criamos os elementos básicos no HTML. Em seguida pegamos cada elementos que vamos usar no Javascript pelo seu ID, ou seja, a lista onde ficará os itens, o input que receberá a entrada do usuário, e o botão que enviará o conteúdo para a lista.

Em seguida colocamos um evento do tipo “click” no botão e dentro da função verificamos se o conteúdo do input é diferente de vazio, se for, então existe um conteúdo para ser inserido.

Criamos um elemento “li”, seu texto recebe o valor que está no input e em seguida colocamos esse item na lista. Após tudo isso feito, zeramos o input para que o usuário não precise apagar manualmente para inserir outras coisas.

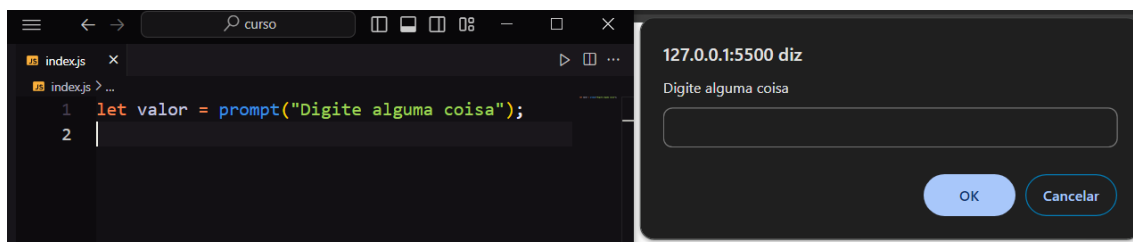
Lista de Desejos

- item 1

Extra

Entrada de forma simples

Com a manipulação do DOM, é possível pegar a entrada do usuário através do uso de inputs como mostrado anteriormente. Porém, existe uma forma mais simples ainda através do método “prompt”, esse método irá gerar uma caixa de texto que recebe uma entrada de usuário:



Quando o usuário inserir o valor e pressionar o botão “OK”, esse valor será armazenado na variável que você criar em Javascript. Mas existe um detalhe, ele **sempre** armazenará o que o usuário inserir como uma **string**, ou seja, caso você queira pegar um número do usuário e fazer operações com esse número, você não vai conseguir por ser do tipo texto.

E pra resolver isso nós temos que converter essa entrada para número usando o “parseInt”

```
let valor = prompt("Digite alguma coisa");  
valor = parseInt(valor);|
```

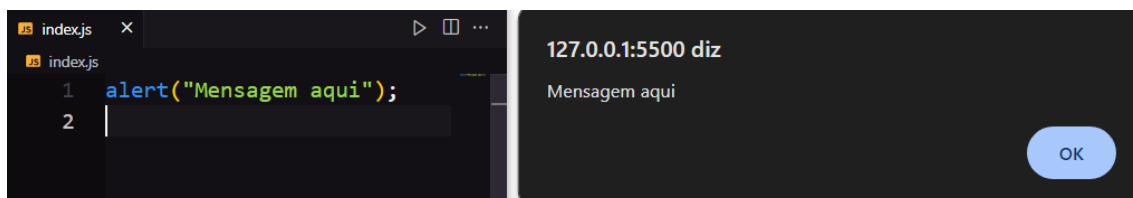
Ou até mesmo:

```
let valor = parseInt(prompt("Digite alguma coisa"));
```

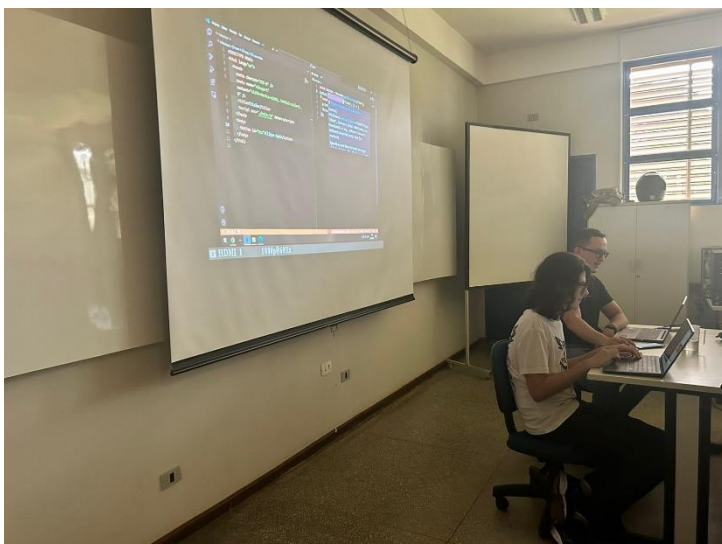
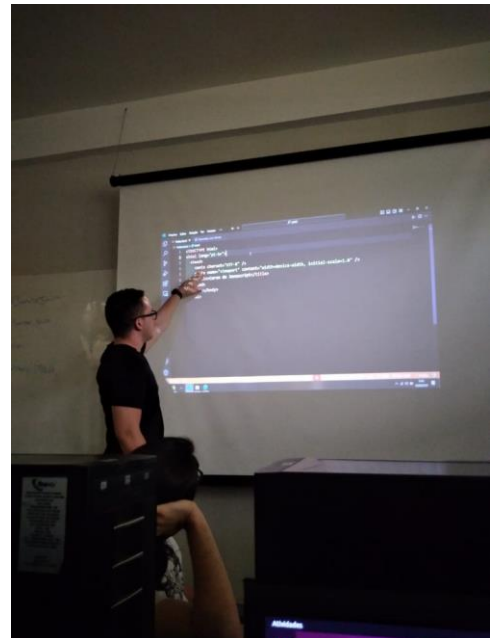
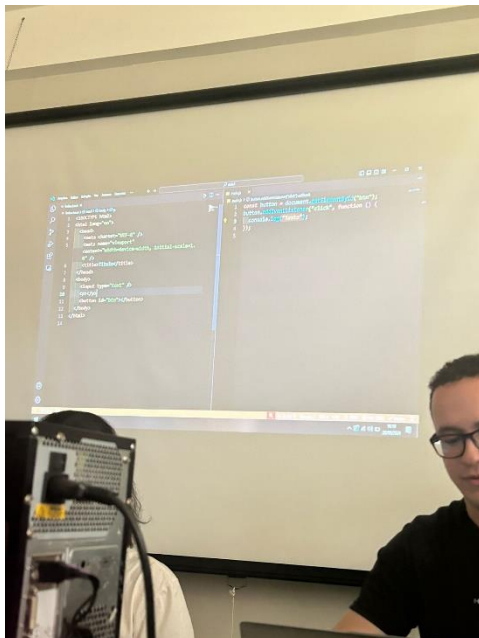
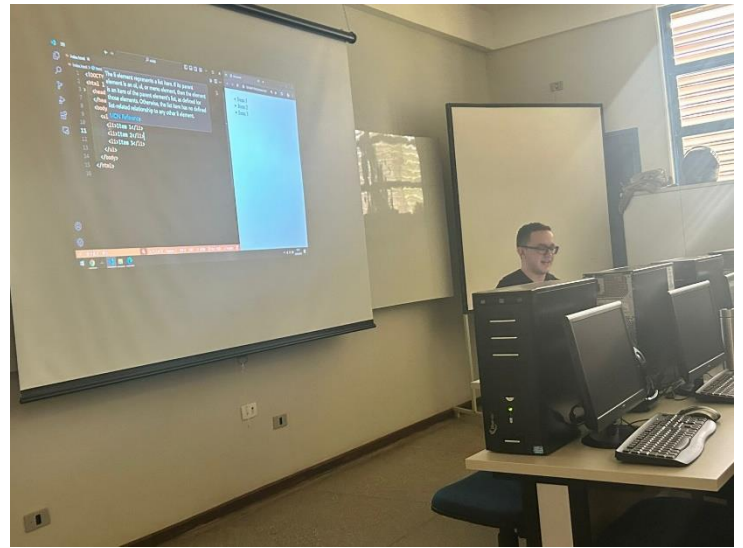
Ele pega a entrada do usuário, converte para inteiro e armazena na variável.

Saída de forma simples

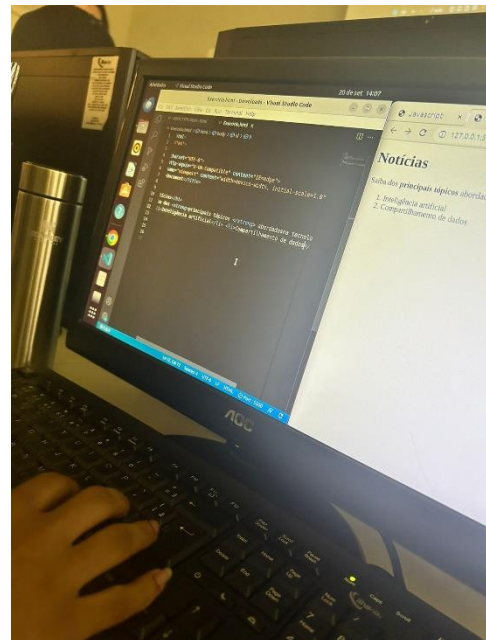
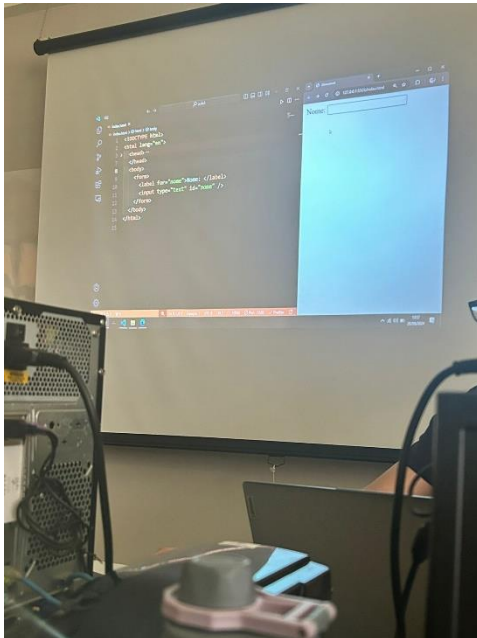
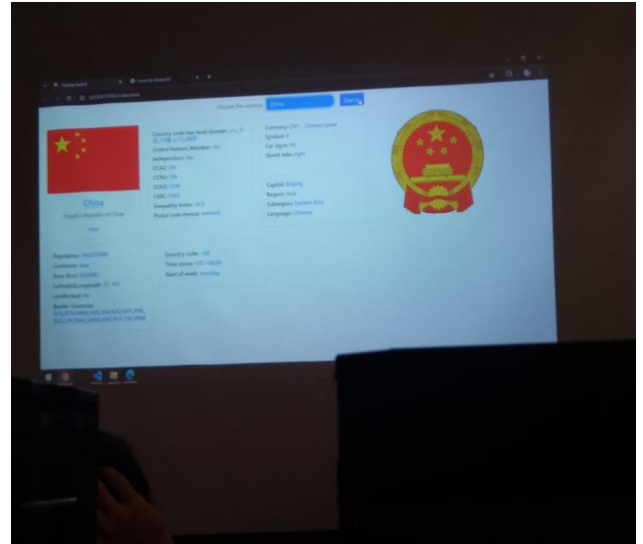
Caso você queira exibir a saída de forma mais simples do que manipular o DOM, você pode usar o console ou o “alert”. Esse método criará um pop-up com o conteúdo que você deseja:



Fotos:



UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL – UEMS
CURSO DE CIÊNCIA DA COMPUTAÇÃO



ABAIXO ARQUIVOS RELACIONADOS A AULA 4, APLICADA DIA 27/09/2024

Em ordem os arquivos previamente planejados.

Em seguida o exercício apresentado e sua resolução, por último as fotos referentes ao dia.

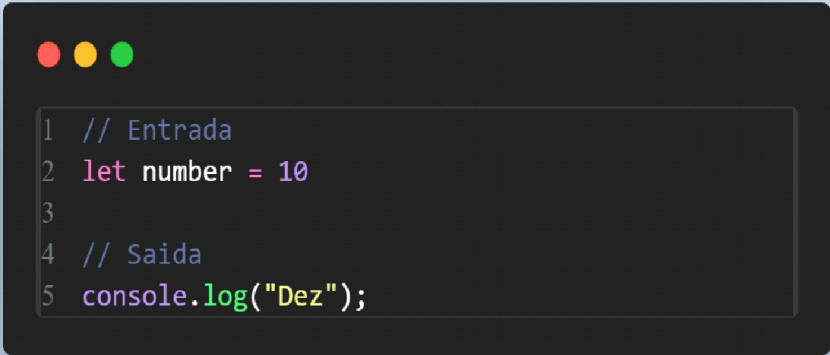
Lista de exercícios

- **Estrutura de decisão**

- Implementar um código que dado a entrada de um número natural “x”, dê a saída de x por extenso.

(x estará apenas na casa das unidades e dezenas [0 á 99])

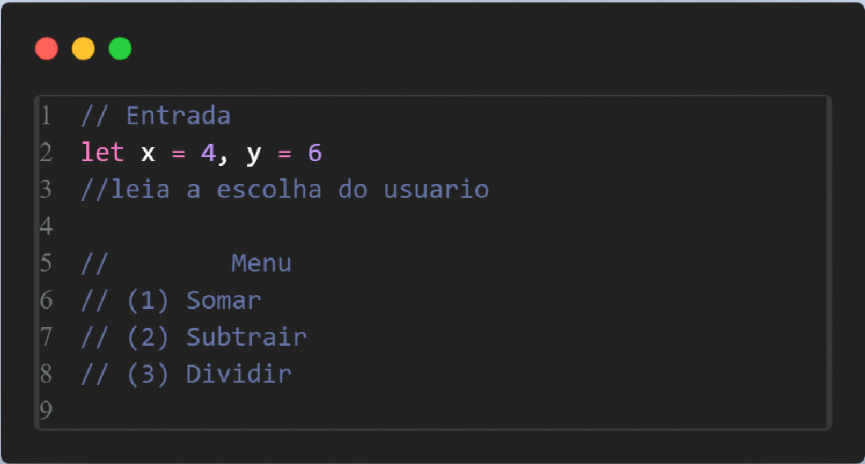
exemplo:



```
1 // Entrada
2 let number = 10
3
4 // Saida
5 console.log("Dez");
```

- **Switch**

- Dada a entrada de dois inteiros X e Y, realizar 3 casos para somar, subtrair e dividir utilizando switch. o programa deve ter um menu visual para escolha do usuário.



```
1 // Entrada
2 let x = 4, y = 6
3 //leia a escolha do usuario
4
5 //      Menu
6 // (1) Somar
7 // (2) Subtrair
8 // (3) Dividir
9
```

- **Função**

- Criar um menu interativo simples com switch e funções, onde entrará 4 notas de números inteiros positivos de (0 até 10). que serão utilizadas para calcular a média, moda e mediana, o programa deve se manter em execução enquanto o usuário utilizar todos os processos devem estar separados em funções !

```

1 // Entrada
2 let notas = [6, 7, 7, 9]
3
4 //      Menu
5 // (1) Calcular media
6 // (2) Encontrar moda
7 // (3) Encontrar mediana
8 // (0) Encerrar
9
10 // Saida
11 " media = 7,25 "
12 " moda = 7 "
13 " mediana = 7 "

```

- **Lógica**

- Faça um programa que receba um cpf e verifica se o formato da entrada é válido.
(entrada através de alertas ou input do tipo texto).
exemplo de entrada valida: **012.456.891-34**

```

1 // calcula tamanho da string
2 (cpf.length == 14)
3
4 // verifica se é um numero
5 // se não for um numero retorna: TRUE
6 // caso contrario: FALSE
7 (isNaN(cpf[i]))

```

Resolução :


```
1 let cpf = prompt("Digite o cpf: ");
2 let verifica = true;
3
4 if (cpf.length === 14)
5 {
6     for (let i = 0; i < 14; i++)
7     {
8         // verifica as posições que deveriam ter
9         // pontos.
10         if ((i === 3) || (i === 7)) {
11             if (cpf[i] !== '.')
12                 verifica = false;
13         }
14         // verifica a posição que deveria ter um
15         // traço.
16         else if (i === 11) {
17             if (cpf[i] !== '-')
18                 verifica = false;
19         }
20         // caso não seja nenhuma posição de
21         // caracteres especiais verifica se
22         // não é um número.
23         else if (isNaN(cpf[i])) {
24             verifica = false;
25         }
26     }
27 } else
28     verifica = false;
29
30
31 if (verifica === true)
32     alert("CPF válido");
33 else
34     alert("CPF inválido");
```

Exercicio Final

- Complementando o exercício anterior, após verificar o formato iremos calcular o cpf. Os 9 primeiros dígitos são utilizados para realizar um cálculo que dará o resultado dos dois últimos chamados de **dígitos verificadores**.

Para exemplos usaremos o cpf **123.456.789-09** ;

Cálculo do 1º dígito verificador:

1. Multiplique cada um dos 9 primeiros dígitos pela sequência de números decrescentes de 10 até 2.
2. Some os resultados das multiplicações.
3. Divida a soma por 11.
4. O resto da divisão (*módulo*) determina o 1º dígito verificador:
 - a. Se o resto for menor que 2, o 1º dígito verificador será 0.
 - b. Se o resto for 2 ou mais, o dígito verificador será **11 - resto**.

Exemplo :

1. $(1 * 10) + (2 * 9) + (3 * 8) + (4 * 7) + (5 * 6) + (6 * 5) + (7 * 4) + (8 * 3) + (9 * 2)$
2. soma == 210
3. divisão $210 / 11 == 19$
4. resto $210 \% 11 == 1$
 - a. se (resto < 2) : (1º dígito == 0) **(o 1º dígito será 0)**
 - b. senão: (1º dígito == **(11 - resto)**)

Calcule o 2º dígito verificador:

1. Agora, considere os primeiros 9 dígitos **mais o 1º dígito verificador** (totalizando 10 dígitos).
2. Multiplique cada um dos 10 dígitos pela sequência de números decrescentes de 11 até 2.
3. Some os resultados das multiplicações.
4. Divida a soma por 11.
5. O resto da divisão determina o 2º dígito verificador:
 - a. Se o resto for menor que 2, o 2º dígito verificador será 0.
 - b. Se o resto for 2 ou mais, o dígito verificador será **11 - resto**.

Exemplo :

1. $(1 * 11) + (2 * 10) + (3 * 9) + (4 * 8) + (5 * 7) + (6 * 6) + (7 * 5) + (8 * 4) + (9 * 3) + (0 * 2)$
2. soma == 255
3. divisão $255 / 11 == 23$
4. resto $255 \% 11 == 2$
 - a. se (resto < 2) : (1º dígito == 0)
 - b. senão: (1º dígito == **(11 - resto)**) **(o 2º dígito será 9)**

```
1
2 let cpf = [], verificadores = []; // strings
3 let soma, divisao, resto; // calculos
4 let i, aux; // operadores
5
6 if (valido == true)
7 {
8     // aloca os digitos em um novo vetor valor numerico
9     for (i = 0; i < cpf_char.length; i++)
10    {
11        if (!isNaN(cpf_char[i]))
12            cpf.push(parseInt(cpf_char[i]));
13    }
14
15    // retira os dois ultimos digitos do vetor em ordem decrescente
16    verificadores.push(cpf.pop())
17    verificadores.push(cpf.pop())
18
19    // soma todos os digitos com seus multiplicadores
20    i = 10
21    soma = cpf.reduce((soma, digito) => soma + digito * i--, 0)
22
23    // calcula a divisao e modulo de 11
24    divisao = soma / 11
25    resto = soma % 11
26
27    // calcula o 1º digito verificador
28    if (resto < 2)
29        cpf.push(0);
30    else
31        cpf.push(11 - resto);
32    // ! 11- resto assim sempre sobrara uma unidade !
33
34    // verificação se continuará a operação para o 2º digito
35    if(cpf[cpf.length-1] == verificadores.pop())
36    {
37        i = 11
38        soma = cpf.reduce((soma, digito) => soma + digito * i--, 0)
39        divisao = soma / 11
40        resto = soma % 11
41
42        if (resto < 2)
43            cpf.push(0);
44        else
45            cpf.push(11 - resto);
46
47        if(cpf[cpf.length-1] != verificadores.pop())
48            valido = false
49    }
50    else
51        valido = false
52 }
53 if (valido)
54     alert("CPF válido");
55 else
56     alert("CPF inválido");
57
```

Fotos:

